SDLog: A Deep Learning Framework for Detecting Sensitive Information in Software Logs

ROOZBEH AGHILI, Polytechnique Montreal, Canada
QIAOLIN QIN, Polytechnique Montreal, Canada
XINGFANG WU, Polytechnique Montreal, Canada
TING-HSUAN LI, Polytechnique Montreal, Canada
FOUTSE KHOMH, Polytechnique Montreal, Canada
HENG LI, Polytechnique Montreal, Canada

Software logs are messages recorded during the execution of a software system that provide crucial run-time information about events and activities. Although software logs have a critical role in software maintenance and operation tasks, publicly accessible log datasets remain limited, hindering advances in log analysis research and practices. The presence of sensitive information, particularly *Personally Identifiable Information (PII)* and *quasi-identifiers*, introduces serious privacy and re-identification risks, discouraging the publishing and sharing of real-world logs. In practice, log anonymization techniques primarily rely on regular expression patterns, which involve manually crafting rules to identify and replace sensitive information. However, these regex-based approaches suffer from significant limitations, such as extensive manual efforts and poor generalizability across diverse log formats and datasets. To mitigate these limitations, we introduce SDLog, a deep learning-based framework designed to identify sensitive information in software logs. Our results show that SDLog overcomes regex limitations and outperforms the best-performing regex patterns in identifying sensitive information. With only 100 fine-tuning samples from the target dataset, SDLog can correctly identify 99.5% of sensitive attributes and achieves an F1-score of 98.4%. To the best of our knowledge, this is the first deep learning alternative to regex-based methods in identifying sensitive information in software logs.

CCS Concepts: • **Security and privacy → Data anonymization and sanitization**; • **Software and its engineering → Operational analysis**; • **Computing methodologies → Neural networks**;

Additional Key Words and Phrases: Software logs, Data privacy, Anonymization, Regular expressions, Deep learning

1 Introduction

Software logs are messages recorded during the execution of a software system, providing crucial run-time information about events and activities. Typically created through logging commands such as *logger.info()*, log message can include different components depending on the logging configuration. Figure 1 shows a log statement generated using Python's *logging* module and contains four components: a timestamp indicating when the event occurred (e.g., 2025-03-26 08:53:29,653), a log level representing the message's severity (e.g., INFO), the

Authors' Contact Information: Roozbeh Aghili, roozbeh.aghili@etud.polymtl.ca, Polytechnique Montreal, Montreal, Canada; Qiaolin Qin, qiaolin.qin@etud.polymtl.ca, Polytechnique Montreal, Montreal, Canada; Xingfang Wu, xingfang.wu@etud.polymtl.ca, Polytechnique Montreal, Montreal, Canada; Ting-Hsuan Li, ting-hsuan.li@etud.polymtl.ca, Polytechnique Montreal, Montreal, Canada; Foutse Khomh, foutse.khomh@etud.polymtl.ca, Polytechnique Montreal, Montreal, Canada; Heng Li, heng.li@etud.polymtl.ca, Polytechnique Montreal, Montreal, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/7-ART
https://doi.org/10.1145/3833386

Just Accepted

```python
def build_log_url(base_url, container_id, log_file):
    url = f"{base_url}/containerlogs/{container_id}/{log_file}"
    logging.info(f"The URL for getting the log: {url}")
    return url
```

↓

```
2025-03-26 08:53:29,653 - INFO - build_log_url - The URL for getting the
log: https://example.com/api/v1/containerlogs/container-123/application.log
```

Fig. 1. A logging statement example in Python.

function name that produced the log (e.g., *build_log_url*), and the message content, which forms the unstructured part of the log. Software logs are essential for developers, operators, and analysts to understand application execution [38], diagnose system failures [15, 66], detect anomalies [45, 78], and monitor systems [7, 46]. In many real-world deployments, logs remain often the only resource for understanding and resolving complex system challenges [7].

Modern software systems are complex, large-scale, and generate huge volumes of logs (e.g., 120-200 million lines per hour [61]). Despite this substantial data volume and the extensive research focused on various aspects of software logs [9, 41], publicly accessible log datasets remain limited. Several studies have highlighted this shortage and called for more open datasets [1, 3, 11, 101]. Although initiatives such as LogHub [101] and ITA [43] have attempted to address this gap, privacy concerns continue to discourage organizations from sharing their production logs.

To share log datasets while protecting privacy, anonymization becomes a critical prerequisite. Before anonymization, practitioners should first identify the target variables and extract them from the log messages. According to privacy regulations such as the *General Data Protection Regulation (GDPR)* and *California Consumer Privacy Act (CCPA)*, sensitive information includes explicit and potentially identifiable (i.e., implicit) information [84]. Based on the regulations, academic literature reviews, and industrial surveys, a recent study identified common sensitive attributes in software logs, which could compromise individual or organizational privacy if exposed [2]. Currently, sensitive attribute detection techniques primarily rely on manually crafted *regular expression (regex)* patterns [21, 52, 83], which require manual effort, struggle to handle unstructured or context-dependent data (e.g., system configurations, usernames), and are prone to the subtle changes in variable formats (e.g., system paths, IP addresses with/without port numbers). As a result, such regex-based approaches face challenges in generalizing to complex logs in real-life scenarios, where sensitive information can appear in diverse and unpredictable formats.

Given the limitations of regex-based sensitive attribute detection, we introduce SDLog: **S**ensitivity **D**etector in Software **Log**s, a deep learning-based framework designed to identify sensitive information in software logs. SDLog is fine-tuned based on the pre-trained *CodeBERT* [20] using sensitive attributes extracted from real-world log data. Our method uses CodeBERT's contextual understanding to address the generalizability limitations of regex-based techniques. Unlike regex, SDLog dynamically interprets surrounding context, making it well-suited for the complex and unstructured real-world software logs. Moreover, SDLog achieves strong performance with minimal labeling effort, requiring only a small number of annotated examples for effective fine-tuning, while demonstrating excellent generalizability across diverse software systems. We validate our approach by annotating and analyzing 32,000 log lines from different software systems, demonstrating high precision in identifying sensitive attributes such as IP addresses, user identifiers, and system paths. We evaluate the effectiveness of our approach by answering the following *Research Questions (RQs)*.

RQ1 ***How effective are regex-based approaches in identifying sensitive attributes in software logs?*** Regular expressions have traditionally been the accepted method for sensitive attribute identification

Just Accepted

in both research and industry. Nevertheless, the effectiveness of regex-based methods across various attribute types and log formats remains largely unexplored. To understand their strengths and weaknesses, we gather regex patterns from academic research, open-source projects, and industry practices, then evaluate their effectiveness on different sensitive attribute types. This systematic evaluation allows us to identify variations, inconsistencies, and coverage gaps across different regex patterns.

RQ2 ***How effective is SDLog in identifying sensitive attributes in unseen software logs?*** SDLog aims to address the generalizability challenges in regex-based approaches by mining contextual and structural information using a language model (i.e., CodeBert). To demonstrate SDLog's effectiveness and generalizability, we conduct a comprehensive evaluation across 16 diverse log datasets, benchmarking SDLog against several baseline approaches, including the best-performing regexes from RQ1, Presidio, Llama3, and GPT-5. Our evaluation consists of two parts: first, we assess SDLog's overall ability to distinguish sensitive attributes from non-sensitivities in software logs; second, we analyze SDLog's performance in accurately identifying specific categories of sensitive attributes, such as IP addresses, MAC addresses, and file paths.

RQ3 ***How effective is SDLog in identifying sensitive attributes when fine-tuned on a target dataset?*** In a real-world scenario, an organization might want to fine-tune SDLog using a subset of its log datasets to tailor the model's sensitive attribute detection to their specific data characteristics. In this research question, we explore how fine-tuning SDLog with a subset of logs from the target dataset can improve its sensitive attribute detection performance. To measure the performance of SDLog in this scenario, we evaluate the fine-tuned SDLog using the same evaluation procedure as in RQ2, analyzing both the sensitive attribute detection of SDLog as well as its performance on the specific categorization of each sensitive attribute.

Our work makes several contributions:

(1) We provide a systematic analysis of the effectiveness of regex-based approaches in detecting sensitive attributes in software logs.
(2) We annotate a dataset of 32,000 log entries labeled with sensitive information categories.
(3) We introduce SDLog, a novel method for identifying sensitive attributes in software logs with minimal fine-tuning efforts and high generalizability.

The rest of the paper is organized as follows. Section 2 summarizes prior research related to our work. Section 3 describes the experiment setup of our study, including the data preparation and the implementation details of SDLog. Section 4 presents our methodology and results addressing the research questions. Section 5 explores the implications of our findings. Section 6 discusses the threats to validity, and finally, Section 7 concludes our paper. We share our replication package information in Section 8.

2 Background and Related Work

2.1 Sharing Software Logs

Although publicly available software logs are scarce, several initiatives have attempted to address this gap. One notable effort is LogHub [101], which is a collection of 16 datasets spanning six categories: distributed systems, supercomputers, operating systems, mobile systems, server applications, and standalone software. Widely adopted in the software engineering community, LogHub serves as a key dataset for research in several domains such as log parsing, anomaly detection, and system monitoring [6, 73]. The *Internet Traffic Archive (ITA)* [43] is another repository that offers server logs and packet traces. It includes large, real-world datasets such as *WorldCup98*, which records the 1.3 billion requests made to the servers of FIFA during the 1998 World Cup, or *Calgary*, which records the requests sent to the department of computer science of the University of Calgary in Canada over one year.

Just Accepted

Although existing open-source data have largely benefited log analysis research, many of these datasets are outdated. For instance, the *Apache* and *BGL* datasets date back to 2005, and all ITA datasets were collected between 1989 and 2007 [3]. Since software usage patterns evolve consistently over time, experiments based on these datasets may lead to conclusions that fail to represent current patterns. This misalignment calls for up-to-date new datasets to be shared in the research community. Sensitive information detection and anonymization are essential methods to mitigate the privacy concerns and encourage log file sharing; our research proposes a novel approach to detecting sensitive attributes in log files.

2.2 Sensitive Information Protection

Sensitive, private information is a major concern in public data sharing. Many regulations and laws are designed to provide guidelines on how organizations collect, store, and use sensitive information, aiming to protect and ensure responsible and ethical handling of sensitive information in different types of data. For example, in the United States, the *Health Insurance Portability and Accountability Act (HIPAA)* [70] addresses healthcare data, while the *California Consumer Privacy Act (CCPA)* [69] focuses on consumer-related information. Apart from general guidelines, several regulations, such as the GDPR and the HIPAA, provide formal definitions of specific attributes that need protection. The attributes can be directly sensitive (e.g., names and email addresses) or serve as quasi-identifiers to indirectly suggest the identity: a well-known study by Sweeney et al. [81] demonstrated that using just three attributes of postal code, date of birth, and sex, over 86% of U.S. citizens could be uniquely identified. While these quasi-identifiers do not directly reveal sensitive information on their own, they can be combined with other data to identify an individual.

Many of these direct or indirect identifiers are often present in software logs, such as timestamps, IP addresses, device or system identifiers, and URLs. The existence of these attributes raises the concern of exposing *Personally Identifiable Information (PII)* (i.e., data that can be used to identify an individual) during software log sharing. Although data privacy has been extensively studied for decades, the particular challenges of preserving privacy in software logs have received limited attention. While data protection laws, such as GDPR and HIPAA, provide guidance on what constitutes personal data, none are tailored specifically to the context of software logs. As a result, existing research on log privacy and anonymization [27, 59, 94] often focuses on a limited, under-explained set of sensitive attributes such as IP and MAC addresses, while largely overlooking the role of other quasi-identifiers within the context of software logs, such as system configurations.

To interpret and adapt relevant regulatory elements, Aghili et al. [2] conducted a systematic study combining literature reviews and surveys of researchers and practitioners to understand which attributes are considered sensitive in software logs. Based on the investigation, they concluded a selection of the most concerning sensitive attributes in software logs, including **IP addresses, MAC addresses, host names, file paths, IDs, URLs, usernames, port numbers**, and **configuration details**. In this study, we adopt their definition and attempt to identify these sensitive attributes in software logs using a machine learning approach.

2.3 Sensitive Information Detection and Anonymization in Software Logs

Preserving privacy in software logs is not just a technical challenge but a fundamental ethical and legal responsibility; it mitigates security risks, protects business confidentiality, maintains user trust, ensures compliance with internal and external policies, and facilitates responsible data governance. The practice of log privacy preservation usually consists of two sequential steps: sensitive attribute detection and anonymization.

Many data anonymization studies predominantly concentrated on IP addresses [2]. One of the earliest and most influential works on log anonymization is CryptoPAn by Xu et al. [94], which enables prefix-preserving anonymization of IP addresses. In this scheme, if two IP addresses share a k-bit prefix in their original form, their anonymized versions will retain that same prefix, preserving the hierarchical structure of the network. Several

Just Accepted

subsequent tools, such as AFT-Anon [30], PD-PAn [27], and the work by Manocchio et al. [55], also focus solely on anonymizing IP addresses with a prefix-preserving strategy, as removing the hierarchical structure of IP addresses can significantly reduce the utility of the data. To address the privacy limitations of prefix-preserving techniques, more recent methods have introduced additional anonymization layers. For instance, AFT-Anon [30] performs real-time anonymization using an in-memory *Active Flow Table (AFT)*, which maps original IPs to their anonymized versions; Varanda et al. [83] apply a *Hash-based Message Authentication Code (HMAC)* built on SHA256 to anonymize IP addresses, leveraging a secret key to enhance privacy and security beyond conventional hashing methods; Xenakis et al. [93] integrates prefix preservation with clustering.

In addition to IP addresses, a few studies proposed anonymization techniques for other attributes, such as host names, file paths, and MAC addresses [2]. One notable example is Flaim [80], which provides a set of anonymization techniques for a wide range of network-related fields, including IP addresses, MAC addresses, host names, port numbers, time stamps, and network protocols. Similarly, iCAT+ [71] offers an anonymization space for IP addresses, integers and decimal numbers, time stamps, and strings. Specifically, it incorporates a *Natural Language Processing (NLP)* model to translate the privacy requirements of data owners and data users. Several other tools also focus on anonymizing network data. TCPdpriv [63] is an anonymization tool designed for processing packet traces via the libpcap library, which is widely used in network traffic analysis. Expanding on the functionalities of TCPdpriv, IP2anonIP [74]allows for the addition of custom fields. Other tools such as AnonTool [22, 23] and PktAnon [25] are tailored for packet-based data, particularly netFlow or libpcap formats. In a similar design, AAPI [42] also works with network data and anonymizes attributes such as IP addresses, TCP and UDP fields, and URLs.

While sensitive attribute anonymization has been widely studied, its prior stage, attribute detection, received inadequate attention. Existing approaches often leverage manually written regular expressions, which are highly static and cannot detect free-format variables such as usernames. As a result, these approaches face challenges in generalizability across datasets [21, 52, 83] and can hinder the anonymization task with unsatisfying performance. To address these challenges, we propose a language model (LM) based approach, SDLog, for highly effective and generalizable sensitive attribute detection.

2.4 Named Entity Recognition (NER)

We model the detection of sensitive information in log messages as a *Named Entity Recognition (NER)* task. Named Entity Recognition is one of the most common token classification tasks in NLP that attempts to find a label for each token in a natural language sequence [47, 65]. Beyond the flat-structure assumption of basic NER, nested NER provides a finer-grained identification by allowing overlapping or hierarchical entities [88]. For example, using blank spaces as the tokenizer, a token may contain an IP address and a port; the two entities together compose a network-related variable. Both types of NER models can be used to identify different sensitive attributes and map a suitable anonymization approach for each attribute. While nested NER is more powerful in representing token attributes, it is less efficient due to layered label inference. The higher time complexity poses a challenge in processing a large amount of log files, making it less suitable for real-life implementations. Therefore, to enable fine-grained sensitive attribute detection (e.g., IP address and port in one network-related variable), we implemented a two-stage basic NER detection structure in SDLog.

A variety of methods have been proposed to tackle the basic NER task. In early approaches, manual engineering is greatly employed to design domain-specific features and heuristic rules [65]. Moreover, several probabilistic models, including *Hidden Markov Models (HMM)* [10], *Maximum Entropy Models (ME)* [12], and *Conditional Random Fields (CRF)* [57] have been employed within supervised learning frameworks to perform this task. Recently, deep learning techniques have become dominant solutions to the NER task and have achieved promising results [47]. Neural NER models leverage deep learning architectures such as *Long Short-Term Memory (LSTM)* [26]

Just Accepted

and *Convolutional neural network (CNN)* [4]to learn contextual representations of tokens. Given the exceptional inference performance of large language models (LLMs), recent researchers also proposed using LLMs such as ChatGPT as an NER solution [87].

A specific type of neural NER model, *Bidirectional Encoder Representations from Transformers (BERT)* [19], is pre-trained on a large corpus and can handle diverse NLP tasks [39]. To meet different needs, many fine-tuned variants have been proposed based on the original BERT model, each introducing architectural modifications or training setups to better suit specific requirements. For example, RoBERTa [53] retains BERT's core architecture but improves performance by enhancing the training process, such as removing the next sentence prediction task and dynamically changing masking patterns. ALBERT [44] enhances BERT's scalability by introducing parameter reduction techniques such as factorized embedding parameterization and cross-layer parameter sharing. ModernBERT [89] introduces a modernized encoder-only architecture that incorporates advancements such as extended context lengths and optimized attention mechanisms, resulting in improving inference efficiency. Specific to coding, researchers fine-tuned BERT with program-related data (e.g., source code and technical discussions) and proposed CodeBERT [20]. This dual understanding of source code and corresponding natural language descriptions allows CodeBERT to perform well in various downstream tasks such as code search [20], code summarization [20], automated program repair [56], and question answering [33]. Similarly, BERTOverflow [82] is fine-tuned on data from Stack Overflow. It incorporates named entity recognition to better handle technical terms and identifiers commonly used in developer discussions, thereby improving its performance on software-related tasks. Considering CodeBERT's strong alignment with coding-related tasks, we fine-tune a CodeBERT backbone by appending a lightweight token-classification layer to enable end-to-end NER without introducing additional task-specific modules.

3 Methodology

In this section, we begin by introducing the sensitive attributes found in software logs, then describe the datasets used for training our model and outline our annotation methodology. Finally, we present the design of our model for identifying sensitive attributes in logs.

3.1 Sensitive Attributes in Software Logs

Software logs contain various attributes, including IP addresses, timestamps, and log levels. As discussed in Section 2.2, this study adopts the definition of sensitivity in software logs proposed by Aghili et al. [2], classifying IP addresses, MAC addresses, host names, file paths, IDs, URLs, usernames, port numbers, and configuration details as sensitive attributes in software logs. These attributes are regarded as the most concerning sensitive attributes or quasi-identifiers according to the literature review and industrial study. Table 1 presents these attributes along with an example for each.

3.2 Benchmark Datasets

We use the log datasets from LogHub-2k [101] to build and test our sensitive attribute detection framework. As shown in Tables 2 and 3, LogHub-2k contains 16 datasets covering different system types, including *distributed systems* (HDFS, Hadoop, Spark, Zookeeper, OpenStack), *supercomputers* (BGL, HPC, Thunderbird), *operating systems* (Windows, Linux, Mac), *mobile systems* (Android, HealthApp), *server applications* (Apache, OpenSSH), and *standalone software* (Proxifier). Each dataset contains 2,000 lines of logs sampled from the corresponding system. Two log messages belong to the same template if they match the same template regex (e.g., "onStandStepChanged 3583" and "onStandStepChanged 3579" both match the template "onStandStepChanged <*>"). Among the 16 datasets, *Apache* contains the fewest log templates with only 6 templates, while *Mac* has the most with 341 templates. The dataset contains 1,358 unique log templates (*Linux* and *Thunderbird* share five common templates),

Just Accepted

Table 1. Types and examples of sensitive attributes in software logs, the attribute examples are highlighted in red.

Attribute	Example
IP Address	*Invalid user webmaster from 173.234.31.186*
MAC address	*ARPT: 621131.293163: wl0: Roamed or switched channel, reason #8, bssid 5c:50:15:4c:18:13, last RSSI -64*
Host name	*proxy.cse.cuhk.edu.hk: 5070 close, 0 bytes sent, 0 bytes received, lifetime 00:01*
File path	*workerEnv.init() ok /etc/httpd/conf/workers2.properties*
ID	*Verification succeeded for blk_-4980916519894289629*
URL	*the url = http://baike.baidu.com/item/%E8%93%9D%E9%87%87%E5%92%8C/462624?fr=aladdin*
Username	*Invalid user webmaster from 173.234.31.186*
Port number	*proxy.cse.cuhk.edu.hk: 5070 close, 0 bytes sent, 0 bytes received, lifetime 00:01*
Configuration details	*mapResourceRequest:<memory:1024, vCores:1>*

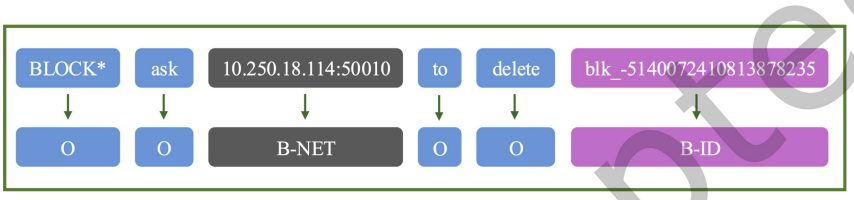

Fig. 2. An example of our data annotation process on the general dataset.

Just Accepted

and 1,363 in total. Although existing studies reported several issues concerning the template qualities (e.g., variables misidentified as a constant) [40], a set of generalizable template correction rules across all studies is yet to be found [76]. To ensure our experimental result is consistent with prior log analysis studies [79, 101], we leveraged the original Loghub-2k labels in our study.

General Dataset. To prepare the data, we preprocess each log message by splitting it into tokens using whitespace as the delimiter, treating each resulting word as a token for sequence labeling. Throughout the remainder of this paper, we refer to this whitespace-splitted dataset with 16 datasets (with 2,000 logs each, totaling 32,000 logs) as the **general dataset**. Although the whitespace tokenization approach works well for most attributes (e.g., URLs, file paths, and IDs), we noticed that network-related information, including IP addresses, host names, and port numbers, often appears concatenated (e.g., *10.250.18.114:50010*) in one token. An approach to address this challenge is to introduce new delimiters (e.g., colons) for their separation. However, the introduction of new delimiters may impact the tokenization of other components (e.g., the colons in a URL entity). As an alternative, we consider such tokens as a *net* category in our general dataset annotation process.

Net Dataset. *Net* is the parent category of IP addresses, host names, and port numbers. To address the two-stage identification task, we derive a specialized subset containing only network-related attributes, referred to as the **net dataset**. Sub-components in a network-related token are separated with special characters. For example, *10.250.18.114:50010* is split into two distinct tokens of *10.250.18.114* and *50010*, which respectively indicate IP address and port number. This design choice mimics a span-based nested-NER task for a single category: the combination of IP addresses, host names, and port numbers constructs a net-type entity. While providing fast identification for other flat-structured attributes, it enables the model to recognize and classify network-related entities even when they are concatenated, ensuring accurate detection despite the limitation of whitespace separation.

3.3 Sensitive Attribute Annotation Process

Our sensitive attribute detection framework relies on fine-tuning a pre-trained language model, CodeBERT. Annotated training data is required for fine-tuning the model to identify sensitive attributes in software logs. We labeled each token with its corresponding attribute in the general and net datasets. The annotation is based on the *IOB (Inside-Outside-Beginning)* format [77], where *B-* denotes the beginning of an attribute, *I-* is used for subsequent words in the attribute, and *O-* indicates non-attribute words. In our annotation task, we assume each token implies independently one type of attribute (i.e., no subsequent token is considered). Hence, we only take *O* (i.e., non-sensitive tokens) and *B* (i.e., sensitive attributes annotated with their category abbreviations) into consideration.

Figure 2 demonstrates the annotation process for a log message from the *HDFS* dataset. In the example, the log message is *BLOCK* ask 10.250.18.114:50010 to delete blk_-5140072410813878235*. For the general labeling process on the whole dataset, we label non-sensitive tokens (i.e., *BLOCK**, *ask*, *to*, and *delete*) as *O*, and the remaining sensitive tokens *10.250.18.114:50010* and *blk_-5140072410813878235* as *B-net* and *B-ID*, respectively. Then, in the net dataset, we label the two sub-components in the *B-net* entity *10.250.18.114:50010* as *B-IP* (*10.250.18.114*) and *B-PORT* (*50010*).

During annotation, two authors (hereafter referred to as annotators) independently performed an open labeling procedure on 200 log templates (roughly 15% of all log templates) randomly selected from all log templates of all datasets. The annotation methodology follows the systematic guidelines proposed by Basili et al. [8]. To ensure labeling reliability, we followed standard practices commonly adopted in similar studies [14, 46, 51]. We adopted a two-round labeling–discussion procedure to progressively align annotator interpretations in the absence of an initial fixed protocol, a common practice when annotating complex and weakly structured artifacts such as software logs [1, 54]. The annotation procedure described below applies to the general dataset. For the net dataset, since the separation task is straightforward, we rely on regular expressions to distinguish IP addresses and host names from port numbers.

Step 1: First Round of Labeling, Discussion, and Revision. To initiate the annotation process, we randomly sampled 100 out of 1,363 templates of the general dataset. Each annotator independently annotated the tokens. During this phase, the annotators did not have a shared annotation protocol. After completing their individual annotations, the annotators shared their responses and discussed their annotation reasoning. The goal of this discussion session was to resolve inconsistencies and establish a shared understanding. Finally, each annotator revised their initial annotations according to the consensus reached.

Step 2: Measuring Reliability for First Round. Establishing reliability is crucial to validate the annotation process [5]. We evaluated inter-annotator agreement using both Cohen's kappa [16] and accuracy, as Cohen's kappa alone can underestimate agreement in imbalanced datasets like ours, where most tokens are labeled as non-sensitive (i.e., annotated as *O*) [29]. After the first round of labeling, the annotators achieved a Cohen's kappa of 0.53 and an accuracy of 0.82, with most disagreements related to the annotation of IDs and configuration details.

Step 3: Second Round of Labeling, Discussion, and Revision. Following the initial round, the annotators conducted another cycle of independent labeling, discussion, and revision on a new random sample of 100 log templates. Their initial annotations in Step 1 were also corrected during this stage. Based on the discussions, a clearer and more consistent understanding of the annotation guidelines has been developed.

Step 4: Measuring Reliability for Second Round. After the second round, we again measured the inter-annotator agreement. The Cohen's kappa improved significantly to 0.87, with an accuracy of 0.94. A Cohen's kappa greater than 0.80 indicates strong reliability in the annotation process [58], confirming that a consistent annotation guideline was successfully established.

Just Accepted

Table 2. Distribution of sensitive attributes in the general dataset. The first number in each cell is the percentage of templates containing the attribute, and the second number in each cell (shown in parentheses) is the total number of attribute occurrences.

Dataset	# Templates	Sensitive	Net	MAC	File path	ID	URL	Username	Config
Android	166	10.2 (257)	0.0 (0)	0.0 (0)	0.6 (7)	9.6 (250)	0.0 (0)	0.0 (0)	0.0 (0)
Apache	6	66.7 (1481)	16.7 (32)	0.0 (0)	33.3 (601)	33.3 (848)	0.0 (0)	0.0 (0)	0.0 (0)
BGL	120	22.5 (175)	6.7 (33)	3.3 (10)	12.5 (124)	3.3 (8)	0.0 (0)	0.0 (0)	0.0 (0)
Hadoop	114	58.8 (2062)	14.0 (1178)	0.0 (0)	5.3 (46)	42.1 (536)	1.8 (6)	0.0 (0)	5.3 (296)
HDFS	14	100.0 (4125)	71.4 (1747)	0.0 (0)	14.3 (378)	100.0 (2000)	0.0 (0)	0.0 (0)	0.0 (0)
HealthApp	75	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)
HPC	46	26.1 (369)	15.2 (173)	0.0 (0)	0.0 (0)	21.7 (196)	0.0 (0)	0.0 (0)	0.0 (0)
Linux	118	28.8 (3865)	7.6 (2356)	0.0 (0)	2.5 (9)	6.8 (1104)	0.0 (0)	2.5 (372)	15.3 (24)
Mac	341	18.2 (443)	5.3 (144)	0.6 (20)	7.3 (199)	1.8 (33)	4.4 (45)	0.0 (0)	0.6 (2)
OpenSSH	27	88.9 (4968)	70.4 (2817)	0.0 (0)	0.0 (0)	22.2 (1009)	0.0 (0)	48.1 (1142)	0.0 (0)
OpenStack	43	95.3 (3559)	16.3 (1040)	0.0 (0)	27.9 (1395)	65.1 (773)	0.0 (0)	0.0 (0)	16.3 (351)
Proxifier	8	100.0 (3042)	100.0 (3042)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)	0.0 (0)
Spark	36	47.2 (1557)	8.3 (3)	0.0 (0)	2.8 (1)	25.0 (1147)	11.1 (77)	0.0 (0)	11.1 (329)
Thunderbird	149	26.8 (955)	6.0 (652)	4.0 (40)	6.0 (81)	2.7 (32)	0.0 (0)	6.0 (109)	9.4 (41)
Windows	50	20.0 (1205)	0.0 (0)	0.0 (0)	10.0 (22)	10.0 (1177)	0.0 (0)	0.0 (0)	2.0 (6)
Zookeeper	50	36.0 (1270)	22.0 (634)	0.0 (0)	6.0 (5)	8.0 (631)	0.0 (0)	0.0 (0)	0.0 (0)
Median	**50**	**32.4 (—)**	**11.2 (—)**	**0.0 (—)**	**5.6 (—)**	**9.8 (—)**	**0.0 (—)**	**0.0 (—)**	**0.0 (—)**
Total	**1363**	**— (29333)**	**— (13851)**	**— (70)**	**— (2868)**	**— (9744)**	**— (128)**	**— (1623)**	**— (1049)**

Step 5: Full Dataset Annotation. Upon achieving a high level of agreement and establishing a clear annotation protocol, the remaining log templates in the general dataset were annotated by the first annotator.

Table 2 presents the detailed final labeling statistics of the general dataset. Each cell reports two values. The first indicates the percentage of templates in the corresponding dataset that contain a given attribute, while the second value represents the total number of occurrences of that attribute within the dataset. For instance, among the 166 templates in the *Android* dataset, 9.6% of them include an ID. The dataset contains 250 ID instances in total. The proportion of sensitive attributes varies significantly: *HealthApp* contains no sensitive attributes, whereas every template in *HDFS* and *Proxifier* includes sensitive attributes. Overall, based on the median across all datasets, approximately one-third of the log templates (32.4%) contain sensitive attributes. Among the different types of sensitive information, net (i.e., IP addresses, host names, and port numbers) is the most common, appearing in 11.2% of log templates in the median. It is followed by ID and file path attributes, with median percentages of 9.8% and 5.6%, respectively. The remaining attributes (i.e., MAC addresses, URLs, usernames, and configuration details) have a median percentage of 0, indicating that less than half of the datasets contain these attributes. For instance, although 48.1% of *OpenSSH* templates contain usernames, 13 out of the 16 datasets have no usernames at all. When considering the total occurrences of sensitive attributes in the general dataset, their distribution is highly imbalanced. Out of 29,333 sensitive attribute instances, 13,851 correspond to network-related attributes. In contrast, MAC addresses and URLs are severely underrepresented, with only 70 and 128 instances, respectively.

Table 3 presents the detailed final labeling statistics of the net dataset. Similar to Table 2, each cell reports two values. The first value indicates the percentage of templates in the corresponding dataset that contain a given attribute, and the second value represents the total number of occurrences of that attribute within the dataset. For instance, in the *BGL* dataset, 6.7% of the 120 templates include an IP address, and it contains 33 IP instances in total. As expected, the total number of attributes in the net dataset is higher than the number of net attributes in the general dataset, as each net attribute in the general dataset could be divided into several attributes (e.g., IP address and port number) in the net dataset. Based on both median and total counts, IP addresses are the most frequent attribute in the net dataset, followed by port numbers and host names. As observed in Table 2,

Just Accepted

Table 3. Distribution of sensitive attributes in the net dataset. The first number in each cell is the percentage of templates containing the attribute, and the second number in each cell (shown in parentheses) is the total number of attribute occurrences.

Dataset	# Templates	IP	Port	Host name
Android	166	0.0 (0)	0.0 (0)	0.0 (0)
Apache	6	16.7 (32)	0.0 (0)	0.0 (0)
BGL	120	6.7 (33)	3.3 (23)	0.0 (0)
Hadoop	114	7.9 (495)	11.4 (1127)	9.6 (1168)
HDFS	14	71.4 (1747)	64.3 (1295)	0.0 (0)
HealthApp	75	0.0 (0)	0.0 (0)	0.0 (0)
HPC	46	8.7 (93)	0.0 (0)	6.5 (80)
Linux	118	7.6 (2356)	0.0 (0)	1.7 (1260)
Mac	341	4.7 (92)	1.8 (71)	0.0 (0)
OpenSSH	27	70.4 (1739)	37.0 (993)	11.1 (471)
OpenStack	43	7.0 (1017)	0.0 (0)	9.3 (23)
Proxifier	8	25.0 (42)	100.0 (3022)	100.0 (3000)
Spark	36	0.0 (0)	5.6 (2)	2.8 (1)
Thunderbird	149	5.4 (641)	0.7 (1)	0.7 (10)
Windows	50	0.0 (0)	0.0 (0)	0.0 (0)
Zookeeper	50	22.0 (635)	22.0 (634)	0.0 (0)
Median	**50**	**7.3 (—)**	**1.25 (—)**	**0.35 (—)**
Total	**1363**	**— (8922)**	**— (7168)**	**— (6013)**

the attribute distribution is highly imbalanced across datasets. For example, while all templates in *Proxifier* contain port numbers and host names, none of the *Android* templates include these attributes. These observations highlight the need for a generalizable sensitive attribute detection pipeline that is capable of handling various attributes across diverse datasets.

3.4 SDLog: Sensitivity Detector in Software Logs

We propose SDLog, a deep learning-based framework designed to detect sensitive attributes in software logs. Since the task of sensitive attribute identification is formulated as an NER task, SDLog can determine whether a word in a log message corresponds to sensitive information and, if so, assign it to the appropriate category (e.g., username). Researchers and developers can leverage SDLog as an alternative to regular expression-based approaches for identifying sensitive data that needs to be anonymized.

SDLog leverages CodeBERT, a pre-trained large language model released by Microsoft [20], to provide generalizable, context-aware sensitive attribute detection. The selection of CodeBERT is based on our justification in Section 2.4, that the model is specifically fine-tuned with coding-related data and optimized to handle syntax, identifiers, and structural elements commonly found in code and, by extension, in log messages. Thus, it is expected to provide a more accurate detection result in comparison to other BERT alternatives. The design and implementation details of SDLog are described in the following sections.

SDLog Hierarchy. Figure 3 presents an overview of our approach to identifying sensitive attributes in software logs. Aligned with our task formulation (i.e., a general classification task and a network-related classification task), SDLog consists of two specialized pre-trained transformer-based models, both fine-tuned from the same pre-trained CodeBERT backbone. **The first model** (general model) is fine-tuned on the general dataset, which is annotated with seven distinct categories of sensitive information: net, MAC, file path, ID, URL, username, and configuration details. This model is trained using the general dataset, enabling it to generalize across a broad range of attribute types and structural patterns observed in real-world logs. **The second model** (net model) is specifically fine-tuned with the net dataset to perform fine-grained classification within the broader net category.

Just Accepted

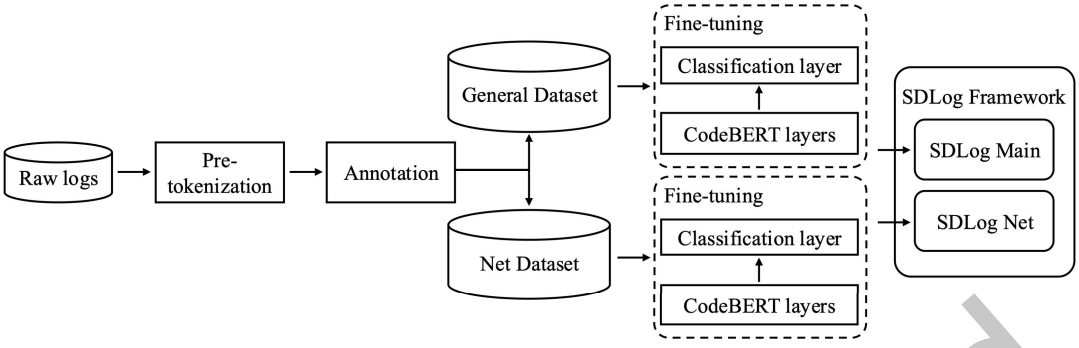

Fig. 3. The overall structure and creation process of the SDLog framework.

This model is trained exclusively on the net dataset. It learns to distinguish between network-related entities, including IP addresses, port numbers, and host names.

This hierarchical detection approach allows SDLog to first identify coarse-grained sensitive attributes and then apply a more targeted classification to tackle complex or overlapping attribute types (e.g., the concatenated IP addresses and port numbers such as "192.168.1.1:8080"). Both models follow the same training procedure by sharing the same hyperparameters and hardware settings.

Classification Head. A single fully connected layer is attached to the output representations of the CodeBERT structure to project each token vector onto the OB label space. A softmax layer then converts these mappings into probability distributions over the label classes. The entire model, including both the classifier and the underlying encoder, is fine-tuned end-to-end using a cross-entropy loss computed on the ground-truth token-level annotations.

Hyperparameter Settings. The initial CodeBERT model is initialized from the *microsoft/codebert-base* checkpoint [20] to perform NER over log data. The model is optimized using the AdamW optimizer with a learning rate of 5e-5 and a weight decay of 0.01. Considering the size of the log dataset, training is conducted for 2 epochs with a linear learning rate scheduler. Evaluation is performed every 300 training steps, and checkpoints are saved at the same interval. To promote generalization and avoid overfitting, the best-performing model, based on validation loss, is retained at the end of training. These hyperparameter choices follow standard practices for adapting pre-trained language models to token-level classification tasks [35].

Hardware Environment. All experiments are conducted on a workstation equipped with an 11th Gen Intel Core i7-11700K CPU (8 cores, 16 threads, up to 5.0 GHz), 32 GB of RAM, and an NVIDIA GeForce GTX 1060 GPU with 3 GB of memory. [1] The system runs a 64-bit Linux environment with Python 3.12, and all models are trained using PyTorch [72] and the Hugging Face Transformers library [91].

4 Experiments and Results

4.1 RQ1. How effective are regex-based approaches in identifying sensitive attributes in software logs?

4.1.1 Approach. We perform a comprehensive search across three research databases, initially identifying 185 relevant articles. We review each article and its replication package (if available) according to inclusion and exclusion criteria to extract regex patterns that identify sensitive attributes in software logs. In addition, we collect

[1]Even with the small memory, no out-of-memory issues were encountered during training or inference.

Just Accepted

regex patterns from three industry partners (a Canadian federal government organization operating a cluster of high-performance computers and two tech companies in the telecommunications domain with 10k - 100k employees), which they use in their anonymization pipelines. Our selection process adheres to the systematic methodology proposed by Wohlin et al. [90] and is detailed below.

Searching Research Databases. Our literature search is conducted across three widely recognized academic databases: IEEE Xplore Digital Library[2], ACM Digital Library[3], and Engineering Village[4], which itself aggregates three databases of Compendex, Inspec, and Knovel. We aim to identify conference and journal papers that use regex patterns for detecting sensitive attributes in software logs. To achieve this, we formulate the following search query:

("Abstract": log) AND ("Abstract": privacy OR "Abstract": sensitiv OR "Abstract": anonymiz* OR "Abstract": log pars*) AND ("Full Text": regex OR "Full Text": "regular expression")*

Apart from the regular expression and sensitive attribute keywords, we also include "log pars*" as a keyword because many log parsers use regex patterns for variable identification during preprocessing [31, 75]. Applying this query, we find 133 articles from the IEEE Xplore Digital Library, 47 from the ACM Digital Library, and 38 from Engineering Village, amounting to a total of 218 articles. After removing duplicate articles, we get a total of 185 papers for further analysis.

We exclude articles that focus on data types other than software logs (e.g., search logs) and focus on identifying papers that use regex to detect one or more sensitive attributes discussed in Section 3.1. We review tables, figures, and code snippets of each paper, searching for keywords such as "regex" and "regular expression". Additionally, we check whether the article includes a replication package. If available, we review it and look for signs of regex patterns, such as the presence of "import re" in Python projects.

Collecting Industry Regexes. In addition to the academic literature, we also collect regex patterns from three of our industry partners. These partners are large-scale organizations with 10k - 100k employees, where they manage extensive log datasets derived from supercomputers, operating systems, and server application environments. The regexes provided by these partners are part of their anonymization pipelines, designed to detect sensitive attributes within their software logs prior to data sharing, either internally or externally.

Inclusion and Exclusion Criteria. Each article, replication package, or industry partner may provide regex patterns for one or more log attributes. We extract regex patterns from research papers, their replication packages, and industry partners, organizing them by attribute type (e.g., IP address and URL). Afterwards, we apply inclusion and exclusion criteria to capture generalizable regex patterns for sensitive attributes. Specifically, we removed regex patterns that identify attributes outside the scope of our defined sensitive attributes (e.g., timestamps) or patterns that are specific to one software system. For example, Li et al. [48] use "blk_-?\d+" to detect block IDs in *Hadoop* logs and "core\.\d+" to identify core IDs in *BGL* logs. We do not consider these regex patterns for the ID attribute, as they are system-specific.

Evaluating Regex Patterns. To systematically evaluate the effectiveness of the extracted regex patterns, we run each regex pattern independently against the appropriate dataset; for example, URL regexes are evaluated on the general dataset, while IP regexes are evaluated on the net dataset. For each execution, we compute standard evaluation metrics including precision, recall, and F1-score, based on token-level ground truth annotations. True positives are counted when the regex-matched text matches the labeled sensitive attributes; false positives are counted when a match occurs outside that sensitive attribute region; and false negatives capture labeled sensitive attributes that were not detected using the regex pattern. Therefore, in this evaluation, precision measures how many of the matches found by the regex patterns were correct, and recall measures how many of the true labeled

[2]https://ieeexplore.ieee.org/
[3]https://dl.acm.org/
[4]https://www.engineeringvillage.com/

Just Accepted

attributes the regex patterns successfully found. This analysis enables a detailed comparison of regex patterns, highlighting their strengths and weaknesses in identifying different types of structured information within heterogeneous logs.

4.1.2 Results.

No common regex pattern exists to identify log sensitive attributes, and their performances vary. Table 4 summarizes the complete set of regex patterns identified through our systematic search. In total, we collected 41 distinct regex patterns across 8 different attribute types, namely IP addresses, MAC addresses, file paths, IDs, URLs, usernames, port numbers, and configuration details. We did not find any regex patterns for host names. Among the found regexes, 17 patterns specifically target IP addresses and are used by 20 different sources. Similarly, for other attributes such as file paths, IDs, URLs, and usernames, each source appears to have developed its own set of regex patterns.

Apart from the variation in design, their performances are also vastly different. For instance, among the regexes for IP addresses, F1-scores range from extremely low values such as 0.0%, 1.1%, and 9.1%, to relatively high values such as 88.4% and 88.5%. Similarly, for regexes designed to detect URLs, we observe F1-scores as low as 0.0%, 1.4%, and 1.8%, alongside patterns achieving F1-scores up to 95.1%.

These results lead to two key observations. First, there is no unified or widely adopted set of regex patterns that researchers and practitioners rely on. Instead, every article, project, or company that requires the identification of sensitive attributes within software logs seems to independently curate and maintain its own collection of regex patterns. Second, different regex pattern designs can lead to vastly different detection performances.

Small differences in regex design have a large impact on effectiveness and generalizability. In our first finding, we noticed that different pattern designs can lead to varied performances. These performance variations can be due to slight differences and details. For example, the two similar regex patterns intended to detect IP addresses, `((\d+).(\d+).(\d+).(\d+))` and `\d+\.\d+\.\d+\.\d+` both aim to match four numeric blocks separated by dots. However, their performance differs considerably. The first pattern achieves a very high recall of 99.7%, and a low precision of 32.5%, indicating that it can successfully detect almost all IP addresses present in the dataset, but also incorrectly identifies many non-IP strings. In contrast, the second pattern yields a high precision of 92.1% and a slightly lower recall of 85.0%, suggesting its capability of labeling true IP addresses in most cases but a slight weakness in true positive coverage. Consequently, the first pattern reaches an F1-score of 49.0%, while the second achieves a much higher F1-score of 88.4%.

We further confirmed our finding by slightly modifying the patterns that resulted in extremely poor detection performance. For example, the sole regex pattern for MAC address assumes that the MAC address must be the only content in the log message, and completely fails to detect any corresponding instance. We deleted the anchors in the pattern as `\b([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})\b` and observe a dramatic improvement, with a precision of 98.8%, recall of 100.0%, and F1-score of 99.4%. Similarly, we increased the F1-score of the seventh IP address pattern to 88.5% by changing it to `\b(25[0-5]|2[0-4]\d|[0-1]?\d?\d)(\.(25[0-5]|2[0-4]\d|[0-1]?\d?\d)){3}\b`.

These examples highlight how small differences in regex structure can significantly impact precision, recall, and overall effectiveness. Further, they showed that regex patterns may lose generalizability across different log structures and datasets due to these subtle details. These findings also suggest the inevitable manual effort and domain knowledge required for regex-based approaches, which hinders their actionability.

While regex patterns can achieve reasonable results in fixed-format attributes, they are not well-suited for detecting free-format attributes (e.g., IDs or usernames). Some regexes are capable of achieving reasonable results (e.g., the best-performing regex patterns in Table 4) for specific types of sensitive attributes, particularly IP addresses and file paths. Their relatively high performance is largely due to the fact that these

Just Accepted

Table 4. Performance of individual regular expressions in categorizing sensitive attributes. All regex patterns are evaluated across all 16 datasets in both the general and net datasets. The highest values of each metric and the regex with the highest F1 scores are highlighted in bold. We mark the indices of the best-performing regex patterns used in RQ2 with a star.

	IP address	P (%)	R (%)	F1 (%)	Source
1	(/|)([0-9]+\.){3}[0-9]+(:[0-9]+|)(:|)	**92.1**	85.1	88.4	[17, 32, 95, 97]
2	([0-9.]+)\s	11.4	48.8	18.5	[100]
3	([0-9]+.){3}[0-9]+(:[0-9]+)	23.0	0.6	1.1	[34]
4	((\d+).(\d+).(\d+).(\d+))	32.5	**99.7**	49.0	[98]
5	(\d)+3\d(:\d+)?	8.6	9.6	9.1	[68]
6	(\d+\.){3}\d+(:\d+)?	**92.1**	85.1	88.4	[60]
7	^(25[0-5]|2[0-4]\d|[0-1]?\d?\d)(\.(25[0-5]|2[0-4]\d|[0-1]?\d?\d)){3}$	0.0	0.0	0.0	[67]
8*	**(\b\d{1,3}(?:\.\d{1,3}){3}\b)**	**92.1**	85.1	**88.5**	[86]
9	**(\d{1,3}(?:\.\d{1,3}){3}):?\d***	**92.1**	85.1	**88.5**	[86]
10	\d+\.\d+\.\d+\.\d+	**92.1**	85.1	88.4	[48]
11	(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})[,:)]	90.7	78.6	84.2	[49]
12	[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}	31.5	**99.7**	47.9	[18]
13	(/|)(\d+.){3}\d+(:\d+)?	32.5	**99.7**	49.1	[75]
14	[0-9]+\.[0-9\.:]*[0-9]	56.7	85.1	68.1	[50]
15	**(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})**	**92.1**	85.1	**88.5**	Company 1
16	\b\d{1,3}(?:\.\d{1,3}){2,}\b	91.8	85.1	88.3	Company 2
17	**(\b\d{1,3}\.)(\d{1,3}\.)(\d{1,3}\.)(\d{1,3}\b)**	**92.1**	85.1	**88.5**	Company 3
	MAC address	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1*	^([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})$	0.0	0.0	0.0	[75]
	File path	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1	((((?<!\w)[A-Z,a-z]:)|(\.{1,2}\\))([^\b%\/|:\n\"]*))|(\"\2([^%\/|:\n\"]*)\")|((?<!\w)(\.{1,2})?(?<!\/)(\/((\\\b)|[^\b%|:\n\"\\\/])+)+\/?)	59.0	98.3	73.7	[49]
2	/[\w/. :-]+	55.6	99.5	71.3	[85]
3	(/[^/\s]+)+	48.1	99.5	64.9	[85]
4	(([A-Z]:)|)(/\S+)+	47.8	99.5	64.5	[95]
5*	**(/|)(([\w.-]+|\<*\>)/)+([\w.-]+|\<*\>)**	**66.7**	98.1	**79.4**	[75]
6	([A-Za-z]:|\.){0,1}(/|\)[0-9A-Za-z\-_\.:/*\+\$#@!\\\?=%&]+(?<![:\.])	48.1	**100.0**	65.0	[50]
7	\/(\S+)	47.8	99.5	64.5	Company 3
	ID	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1	(?:UUID|GUID|version|id)[\\=:\"\'\s]*\b[a-fA-F0-9]{8}-[a-fA-F0-9]{4}-[a-fA-F0-9]{4}-[a-fA-F0-9]{4}-[a-fA-F0-9]{12}\b	0.0	0.0	0.0	[85]
2	<([^>]+)>	2.6	0.1	0.2	[85]
3	[pP]id[:|-|=|\s/]*(\d+)	97.7	1.3	2.5	[49]
4*	**[uU]id[:|-|=|\s/]*(\d+)**	**99.8**	**23.5**	**38.1**	[49]
	URL	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1*	**[A-Za-z\.]+://[A-Za-z0-9\.\/\+#@:_\-]+(?<![:\.])**	91.4	99.2	**95.1**	[50]
2	(https?://\S+)	**100.0**	39.1	56.2	[49]
3	https?://[^\s#]+#[A-Za-z0-9\-\=\+]+	0.0	0.0	0.0	[85]
4	http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[!*\\(\\).]|(?:\%[0-9afA-F][0-9a-fA-F]))+	**100.0**	39.1	56.2	[85]
5	([\w-]+\.)+[\w-]+(:\d+)?	0.9	**100.0**	1.8	[99]
6	(\S+\.\S+(\.\S+)+(:\d+)?)|(\w+-\w+(-\w+)+)	0.7	**100.0**	1.4	[95]
7	\bhttps?://(www.)?[a-zA-Z0-9-]+(.[a-zA-Z]{2,})+(:[0-9]{1,5})?(/[^\s]*)?\b	**100.0**	31.2	47.6	[75]
	Username	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1	user(|)[A-Za-z0-9]+(?<!request)(?! methods)	**42.0**	25.3	31.6	[50]
2	user\:\s(\w+)	0.0	0.0	0.0	[96]
3*	**r?[uU]ser[:|-|=|\s/]*<(\w+)>|r?[uU]ser[:|-|=|\s/]*(\w+)**	35.2	**72.0**	**47.3**	[49]
	Port number	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1*	**[pP]ort[=: |:|=|: |\s/]*(\d1,5)**	**96.0**	**8.1**	**15.0**	[49]
	Configuration details	**P (%)**	**R (%)**	**F1 (%)**	**Source**
1*	**size\s+(\d+)**	**19.2**	**14.2**	**16.3**	[86]

Just Accepted

attributes often follow strict and predictable patterns that remain consistent across a variety of log formats and datasets.

On the other hand, regex patterns generally perform poorly for detecting certain attributes such as IDs, usernames, port numbers, and configuration details. This is primarily due to the absence of a consistent and rigid structure for these attributes, which makes them difficult for regex patterns to capture effectively. For example, usernames may or may not be preceded by the keyword *user*, complicating the task for regex patterns designed with specific assumptions. Similarly, port numbers can be located in various positions within a log; immediately after an IP address, immediately after a host name, or embedded in text. Therefore, few regex patterns are designed for detecting these attributes. As shown in Table 4, only 4, 3, 1, and 1 patterns were found for IDs, usernames, port numbers, and configuration details, respectively, and no pattern is designed for host names. Moreover, such regex patterns often result in low effectiveness. For instance, the best-performing regex pattern for detecting IDs achieved an F1-score of just 38.1%, and the best pattern for detecting usernames had an F1-score of 47.3%. The performances for detecting port numbers and configuration details are even worse, with F1-scores of 15.0% and 16.3%, respectively. Given the detection difficulties, these free-formatted yet important attributes are often overlooked in sensitive attribute protection practices.

Summary of RQ1

Our analysis shows that there is no common ground truth for regex patterns to detect sensitive attributes in software logs; instead, researchers and practitioners tend to develop their own patterns, leading to inconsistency and fragmentation. Regex performance varies significantly even within the same attribute type, with small design differences causing large impacts on precision and recall. While regex can achieve reasonable results for well-structured attributes such as IP addresses, it performs poorly for less structured attributes such as IDs, usernames, port numbers, and configuration details. Overall, regex patterns often lack generalizability across datasets and are not well-suited for attributes without strict formatting.

4.2 RQ2. How effective is SDLog in identifying sensitive attributes in software logs?

4.2.1 Approach. In this section, we evaluate the effectiveness of SDLog in identifying sensitive attributes and compare its results with state-of-the-art baselines.

Baselines. We compare SDLog's performance with four state-of-the-art sensitive attribute detection solutions, including two regex-based approaches and two representative LLMs (i.e., GPT-5 and Llama3-8B) with few-shot prompting. Although the two LLMs are recognized as general-purpose, they have shown exceptional performance in coding-related tasks such as bug fixing and code refinement [13, 28, 64].

- **Regex.** The baseline regex consists of the best-performing (i.e., highest F1-score) regex pattern for each attribute according to the results from RQ1. We randomly choose one of the patterns in cases where multiple regex patterns for the same attribute achieve optimal F1-scores; the selections are marked with "$*$" in Table 4. Notice that we leverage the refined regex in RQ1 for MAC address detection. Using the eight patterns, we design a sequential sensitive attribute detection pipeline that matches the implementation strategy of our industrial partners: each log line is passed through all eight regexes in sequence, and a placeholder will replace the value when it matches a certain pattern (e.g., an IP match is replaced by *$IP*). The sequential process aims to avoid multi-labeling an attribute (e.g., a variable may be identified as both a URL and a file path when the regex patterns are not accurately designed).
- **Presidio.** We include Microsoft Presidio as an industrial-grade baseline that is widely used for data anonymization [62]. Presidio detects sensitive entities using a combination of predefined recognizers,

Just Accepted

primarily based on regular expressions and rule-based validation logic, optionally enhanced with context-aware scoring. Presidio is applied to each log line independently, and its character-level detection results are converted into token-level labels by marking overlapping tokens as positive predictions.

- **GPT-5.** We include GPT-5 to represent prompt-driven, context-aware large language models for detecting sensitive attributes. Unlike regex-based approaches, GPT-5 leverages the surrounding textual context to infer sensitive attributes and does not rely on manually engineered patterns. GPT-5 is claimed to have a stable performance and does not support temperature tuning. To further mitigate model hallucination, we followed the few-shot prompting strategy to demonstrate the labeling format (i.e., label the sensitive attributes on a token-by-token basis) and define the attribute taxonomies.
- **Llama3-8B.** We include Llama3-8B to represent locally deployable, prompt-driven models for sensitive attribute detection. In comparison to GPT-5, Llama3-8B is more lightweight and is more privacy-aware. For hallucination mitigation, we configured Llama3-8B's temperature to 0 and implemented the same few-shot prompting strategy as GPT-5.

Evaluation. Previous research suggested that the order of regex patterns can impact the detection performance [75]. For example, if a loosely written ID regex partially matches an IP address and is executed before the IP regex, the true IP may be incorrectly masked as *$ID*. However, if the IP regex is executed first, it correctly identifies and replaces the IP address. Therefore, for the regex baseline, we randomly sampled 500 orders out of the 40,320 (8!) possible permutations and report their minimum and maximum precision, recall, and F1-score values.

Regarding Presidio, it natively only supports IP addresses and URLs and does not provide out-of-the-box recognition for other log-specific attributes considered in this study. Extending Presidio to cover additional attributes would require incorporating custom rule-based patterns (e.g., regular expressions) or integrating external machine learning or AI-based models. Since such approaches are already represented by our regex and LLM-based baselines, we limit our evaluation of Presidio to the attributes it officially supports under its default configuration. For GPT-5 and Llama3-8B, we applied the same procedure as the regex-based and Presidio baselines: the model processes each log line independently and outputs token-level predictions. The computation of precision, recall, and F1-score for each baseline shares the same token-level evaluation protocol.

To evaluate the effectiveness and generalizability of SDLog, we use cross-validation to train and evaluate SDLog. For the general model, we use *Leave-One-Out Cross-Validation (LOOCV)* [92], training the model on all datasets except one and testing it on the excluded dataset. This procedure is repeated until each dataset has been used as the test set once, resulting in 16 fine-tuned models. We choose this setting as it better reflects real-world deployment scenarios, where models are trained on logs from known systems and applied to previously unseen systems. By conducting LOOCV over our diverse set of logs, we obtain a rigorous and comprehensive evaluation of SDLog's effectiveness in identifying sensitive attributes across varying log formats and sources. For the net model, we use 4-fold cross-validation [24], splitting the 16 datasets into 12 for training and 4 for testing in each round. 4-fold cross-validation is more suitable and balanced than LOOCV for the net model, considering the small size of the net dataset, as three datasets (i.e., *Android*, *HealthApp*, and *Windows*) contain no net attributes. Using 4-fold cross-validation results in 4 fine-tuned models for the net dataset.

We evaluate the general and net models through a two-step process. First, we measure their ability to detect sensitive attributes in datasets that they have not encountered during training. These attributes correspond to the nine categories outlined in Section 3.1. The goal is to determine whether SDLog can accurately locate these sensitive attributes. Second, we evaluate SDLog's ability to classify the detected sensitive attributes into their respective sensitive categories. This two-tiered approach comprehensively evaluates SDLog's generalizability on new data and effectiveness in distinguishing different types of sensitive data.

4.2.2 Results.

Just Accepted

Table 5 summarizes SDLog and baseline approaches' performance per-attribute on both general and net datasets, and Table 6 summarizes their performance per-dataset on the general dataset. The two tables report each tool's precision (i.e., the proportion of correctly predicted sensitive attributes among all predictions), recall (i.e., the proportion of actual sensitive attributes correctly identified), and F1-score (i.e., the harmonic average of precision and recall) on different dimensions. Specifically, we do not include Presidio's results in Table 6, as it only supports the detection of a limited subset of the attributes (i.e., Username and IP addresses). We also exclude the *HealthApp* dataset from Table 6, as it does not contain any sensitive attributes in the entire dataset, and therefore, metrics such as precision, recall, and F1-score become undefined or uninformative.

The order of regex patterns can significantly affect performance per-attribute. A previous study on log preprocessing pointed out that regex's performance in variable detection is related to the orders of the patterns. For example, given their similar structure, MAC addresses can be segmented into two "time-like" variables when they only contain digits and colons (e.g., 00:00:00:00:00:00). As a result, these MAC addresses cannot be identified if time variables are masked before them. The Regex min-max columns in Table 5 present the results of applying our top-performing regex patterns for detecting sensitive attributes in software logs, executed 500 times with randomized pattern orders. Since most of the sensitive attributes do not share similar structures, randomized order does not strongly impact the detection performance on most variables, such as file paths (F1-score ranging from 79.3% to 80.5%) and IP addresses (F1-score ranging from 88.5% to 88.9%). However, the F1-score of URL detection fluctuates drastically, from as low as 16.6% up to 95.1%, representing a 78.5% difference. This is due to URL variables often containing components that are, or similar to, other variables, which corresponds to the conclusion in the existing study. For example, usernames are common in URLs of a user's main page. Following this observation, we empirically determined the optimal ordering of regex patterns by executing the detection pipeline 500 times with randomized pattern sequences and selecting the ordering that achieved the highest performance. The F1-scores of this optimal order correspond to the maximum scores in Table 5. This result highlights two key observations: (1) the sequence in which regex patterns are applied can have a substantial impact on detection performance, (2) when new sensitive attributes are required to be detected in a system, manual analysis on variable structures is required to obtain the optimal ordering, which hinders their scalability to real-world log datasets.

General purpose Large language models have less stable detection performances. Large language models such as GPT-5 and Llama3-8B are trained on a large corpus from various domains. Known for high inference performance and generalizability, they have been widely applied in coding-related tasks and research. However, their performance on log-specific sensitive attribute detection is inconsistent and often unreliable. As shown in Table 5, GPT-5 achieves highly variable F1-scores across attributes, ranging from 0.0% for URL and MAC detection to 78.8% for file paths and 63.4% for IP addresses, indicating a lack of robustness across attribute types. Similarly, Llama3-8B performs even worse in many cases, with F1-scores as low as 2.1% for URL and 4.6% for usernames, and only moderate performance on certain attributes such as IP address (20.6%). This instability is also reflected at the dataset level (Table 6), where GPT-5 ranges from as low as 1.3% on Windows dataset to 80.4% on Proxifier dataset, while Llama3-8B frequently underperforms with near-zero results on multiple datasets (e.g., 0.0% on Windows and 0.1% on Proxifier). In contrast, SDLog consistently achieves high performance across both attributes and datasets, with F1-scores typically above 90% (e.g., 99.7% for IP address and 92.9% overall), significantly outperforming both LLMs and regex-based baselines. These results suggest that, despite their general capabilities, off-the-shelf LLMs are not well-suited for precise and reliable sensitive attribute categorization in log data without task-specific adaptation.

SDLog demonstrates strong performance in categorizing most sensitive attribute types, and its second-level granularity achieves near-perfect categorization of net attributes. According to Table 5, SDLog achieves the highest F1-score on 7 of the 9 attributes in the general dataset and on all 3 attributes in the net dataset. Specifically, it achieves near-perfect performance on file paths (99.9% precision, 94.8% recall),

Just Accepted

Table 5. Comparison of SDLog performance with baseline approaches in categorizing sensitive attributes. The highest F1 values are highlighted in bold.

Attribute	Regex (worst order)			Regex (best order)			GPT-5			Llama3-8B			Presidio			SDLog		
General	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**
MAC	98.6	100.0	**99.3**	98.6	100.0	**99.3**	0.0	0.0	0.0	9.5	11.4	10.4	-	-	-	100.0	40.0	57.1
File path	66.6	97.7	79.3	68.4	98.0	80.5	92.8	68.4	78.8	49.3	19.6	28.1	-	-	-	99.9	94.8	**97.3**
ID	99.8	23.5	38.0	99.8	23.5	38.0	23.4	5.1	8.4	15.1	29.1	19.9	-	-	-	86.0	91.5	**88.7**
URL	70.6	9.4	16.6	94.6	99.2	**95.1**	0.0	0.0	0.0	1.1	24.2	2.1	0.1	40.6	1.3	0.0	0.0	0.0
Username	36.6	72.0	48.5	36.6	72.0	48.5	55.0	76.3	63.9	3.0	9.9	4.6	-	-	-	99.8	73.7	**84.8**
Configuration	19.2	14.2	16.3	19.2	14.2	16.3	0.9	0.3	0.4	5.8	23.6	9.3	-	-	-	95.7	34.2	**50.4**
Net	-	-	-	-	-	-	40.4	96.3	56.9	53.9	32.6	40.6	-	-	-	99.5	97.8	**98.6**
Net	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**	**P (%)**	**R (%)**	**F1 (%)**
IP	92.1	85.1	88.5	93.0	85.1	88.9	76.8	54.1	63.4	15.3	31.5	20.6	87.6	85.6	86.1	100.0	99.4	**99.7**
Port	96.0	8.1	15.0	96.2	8.1	15.0	22.6	38.6	28.5	14.5	32.8	20.1	-	-	-	100.0	100.0	**100.0**
Host name	-	-	-	-	-	-	14.0	80.3	23.9	9.4	12.3	10.6	-	-	-	100.0	99.2	**99.6**

usernames (84.8% F1-score), and configuration details (50.4% F1-score), consistently outperforming all baselines. For more challenging attributes, such as ID, username, and configuration details, which lack consistent structure and vary significantly across datasets, SDLog still achieves F1-scores of 88.7%, 84.8%, and 50.4%, respectively. It also demonstrates particularly strong performance on network-related entities, achieving 99.5% precision for the net category.

At the fine-grained level, the second, lightweight model of SDLog is dedicated to distinguishing between the sub-attributes of the net attribute, namely, IP addresses, port numbers, and host names. Trained only on these attributes, SDLog achieves high F1-scores of 99.7% for IP addresses, 100.0% for Port numbers, and 99.6% for host names. This two-stage approach, first detecting net-related entities, then categorizing the sub-attributes of it, proves effective in handling attributes that frequently appear in similar contexts and overlapping or compound forms (e.g., IP:port or host:port).

On the other hand, baseline approaches have less stable detection performance for different attributes. For instance, although Regex (max) attains high recall (98.0%) for file path identification, its low precision (68.4%) limits its overall F1-score. For the same type of attributes, GPT-5 has a similar performance with an F1-score of 78.8%, while Llama3 reaches only 28.1%. The comparison further highlights a key pattern: regex-based approaches perform well when attributes follow fixed and well-defined formats, such as MAC addresses and URLs. However, for context-dependent attributes without rigid patterns such as usernames, IDs, and configuration details, regex performance drops substantially. For example, Regex achieves F1-scores of only 38.0% for IDs, 48.5% for usernames, and 16.3% for configuration details, whereas SDLog improves these to 88.7%, 84.8%, and 50.4%, respectively. Port number detection further illustrates this contrast. Regex (max) achieves only 8.1% recall and a 15.0% F1-score, and GPT-5 improves this only modestly to 28.5%. SDLog, however, first detects the net category with an F1-score of 98.6%, and subsequently distinguishes port numbers with perfect accuracy (100.0% F1-score). Similarly, host names are detected with near-perfect performance by SDLog, while no effective regex pattern was identified for this attribute.

SDLog has lower performance on MAC and URL attributes due to limited labeled data and tokenization ambiguity. While SDLog achieves strong overall performance, its lower F1-scores on MAC addresses (57.1%) and URLs (0.0%) can be explained by two main factors: limited labeled data and tokenization-induced ambiguity. Across the entire general dataset, only 70 MAC and 128 URL instances are available (please check Table 2). As a result, the number of training examples is very small; for example, when evaluating on the Spark dataset, only 51 URL instances are available in the training set. In contrast, other attributes, such as net and ID, are represented by 13,851 and 9,744 samples, respectively. This scarcity, combined with high structural variability, makes it difficult for the model to learn robust and generalizable representations.

Just Accepted

Table 6. Comparison of SDLog performance with baseline approaches on different general datasets. The highest F1 values are highlighted in bold.

Dataset	Regex (worst order)			Regex (best order)			GPT-5			Llama3-8B			SDLog		
	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)
Android	12.2	7.8	9.5	12.8	20.3	10.8	19.8	68.1	30.6	3.4	31.1	6.1	80.5	89.9	**84.9**
Apache	25.0	25.0	25.0	25.0	25.0	25.0	2.4	4.3	3.1	0.8	0.8	0.8	100.0	100.0	**100.0**
BGL	36.1	37.5	36.8	36.1	37.5	36.8	42.3	56.6	48.4	1.8	16.2	3.3	100.0	86.3	**92.6**
Hadoop	29.9	13.5	14.3	44.3	25.5	22.4	28.6	75.8	41.6	18.3	19.5	18.9	98.8	80.0	**88.4**
HDFS	25.0	25.0	25.0	25.0	25.0	25.0	75.4	61.0	67.4	56.2	51.6	53.8	93.1	95.1	**94.1**
HPC	12.5	12.5	12.5	12.5	12.5	12.5	7.5	25.2	11.5	2.4	3.3	2.8	100.0	68.8	**81.5**
Linux	36.8	43.3	36.0	36.8	43.3	36.0	48.0	78.0	59.5	66.0	67.5	66.8	99.8	99.7	**99.7**
Mac	52.4	42.9	43.0	53.2	52.4	51.4	11.1	39.0	17.3	4.0	22.7	6.7	62.3	49.9	**55.4**
OpenSSH	43.5	39.9	40.6	43.5	39.9	40.6	64.1	74.3	68.8	23.3	21.0	22.1	92.0	91.6	**91.8**
OpenStack	19.7	25.0	21.7	19.7	25.0	21.7	34.3	68.4	45.7	31.0	42.1	35.7	100.0	88.8	**94.1**
Proxifier	12.5	6.5	8.6	12.5	6.5	8.6	67.3	100.0	80.4	0.1	0.1	0.1	100.0	100.0	**100.0**
Spark	37.7	31.0	21.3	50.0	43.2	45.3	0.1	0.3	0.1	11.4	11.6	11.5	62.3	64.2	**63.2**
Thunderbird	66.8	62.4	61.2	66.8	62.4	61.2	71.3	88.6	79.0	33.2	68.6	44.7	97.1	86.5	**91.5**
Windows	3.3	4.5	3.8	3.3	4.5	3.8	1.9	1.0	1.3	0.0	0.0	0.0	99.8	99.0	**99.4**
Zookeeper	22.9	25.0	23.9	22.9	25.0	23.9	80.7	48.9	60.9	16.6	19.5	17.9	99.9	99.8	**99.8**
Macro Average	29.1	26.8	24.7	31.0	29.9	28.3	37.0	52.6	41.0	17.9	25.0	19.4	92.4	86.6	**89.1**
Micro Average	29.4	27.9	26.4	31.0	29.7	28.3	41.2	66.5	51.5	24.2	29.4	26.5	94.6	91.2	**92.9**

In addition, as discussed in Section 3.2, we preprocess each log message by splitting it into tokens using whitespace as the delimiter. This approach is quick and straightforward; however, it can cause some ambiguity for the model. For instance, in the *BGL* log message *PrepareForService shutting down Node card(mLctn(R10-M1-N2), mCardSernum(20323150383334300000000594c31304b34323934303257), mLp(FF:F2:9F:16:E0:DA:00:0D:60:E9:1F:25), mIp(10.1.1.164), mType(4)) as part of Service Action 648* the full token *mLp(FF:F2:9F:16:E0:DA:00:0D:60:E9:1F:25)* is labeled as a MAC address, even though the actual MAC value is embedded within the token. A similar issue arises for URLs, such as in *PBRequester failed with Error ... UserInfo=NSErrorFailingURLStringKey=https://gsp-ssl.ls.apple.com/dispatcher.arpc ... The Internet connection appears to be offline.* where the entire *UserInfo=NSError-FailingURLStringKey=https://gsp-ssl.ls.apple.com/dispatcher.arpc,* is labeled as a URL, while the true URL is only a substring of the token.

Taken together, the combination of tokenization and the limited number of labeled examples explains the lower performance of SDLog on MAC and URL attributes. We expect performance on these attributes to improve substantially with finer-grained tokenization or additional annotated data.

SDLog consistently achieves high detection performance across diverse log datasets (with an average F1-score of 92.9%), significantly outperforming all baselines. As shown in Table 6, regex-based approaches exhibit limited effectiveness, with *Regex (worst order)* and *Regex (best order)* achieving micro-average F1-scores of 26.4% and 28.3%, respectively. This small gap highlights that even with optimal ordering, regex performance remains constrained. Similarly, *Llama3-8B* performs comparably to regex, with an F1-score of 26.5%, indicating its limited suitability for this task. *GPT-5* performs relatively better, reaching an F1-score of 51.5%, yet still falls substantially short of SDLog. In contrast, SDLog achieves a precision of 94.6%, recall of 91.2%, and F1-score of 92.9%, demonstrating a substantial margin of improvement over all baselines, including state-of-the-art large language models.

Notably, SDLog demonstrates strong and stable performance across nearly all datasets. It achieves F1-scores above 80% on 13 out of 15 datasets, with only *Mac* (55.4%) and *Spark* (63.2%) falling below this threshold. In comparison, the strongest baseline, *GPT-5*, exceeds 80% only on the *Proxifier* dataset, highlighting its lack of consistency. Furthermore, SDLog achieves perfect performance (F1-score of 100%) on both the *Apache* and

JUST ACCEPTED

Table 7. Comparison of sensitive attribute detection results on a representative log line.

Token	[client	222.166.160.184]	Directory	index	forbidden	by	rule:	/var/www/html/
SDLog	O	B-NET	O	O	O	O	O	B-PATH
GPT	B-ID	B-NET	B-ID	O	B-ID	O	O	B-PATH
Llama3	B-NET	O	O	O	O	O	B-PATH	B-URL

Proxifier datasets, correctly identifying all sensitive attributes without error. Across all evaluated datasets, SDLog consistently obtains the highest F1-score, underscoring its robustness and superior generalization capability in handling diverse and complex log formats.

A qualitative comparison illustrates how different models label a sample log entry. Table 7 presents a representative log entry along with the token-level predictions generated by SDLog, GPT-5, and Llama3. In this example, SDLog correctly identifies the IP address and file path while leaving all other tokens unlabeled, resulting in a precise and semantically consistent prediction. In contrast, both GPT-5 and Llama3 demonstrate substantial over-detection and label inconsistencies. GPT-5 incorrectly assigns sensitive labels (e.g., B-ID and B-NET) to non-sensitive tokens such as *[client*, *Directory*, and *forbidden*, suggesting an overgeneralization of entity boundaries based on surface-level patterns rather than log-specific semantics. Similarly, Llama3 misclassifies the IP address and incorrectly labels the file path as URL, indicating confusion between structurally similar but semantically distinct attributes. These errors likely arise because general-purpose LLMs rely primarily on natural language context and token co-occurrence patterns, which do not align well with the rigid syntax and domain-specific conventions of software logs.

Summary of RQ2

The results show that SDLog consistently outperforms all baseline approaches in detecting sensitive attributes in software logs. Regex-based methods struggle particularly with attributes lacking a fixed structure, while general-purpose LLM-based methods are not well-tailored for tasks in strictly coding contexts. In contrast, SDLog excels in handling attributes without predefined structure, such as username, ID, and configuration details, highlighting its effectiveness and generalization capability in real-world log datasets. It achieves robust and accurate sensitive attribute detection across diverse datasets, with an overall F1-score of 92.9%, significantly outperforming all baselines.

4.3 RQ3. How effective is SDLog in identifying sensitive attributes when fine-tuned on a target dataset?

4.3.1 Approach. RQ2 evaluates SDLog's generalizability on unseen data. Instead of directly using the model pretrained on other log datasets, practitioners may sample a small batch of labeled logs from their target dataset and further fine-tune the model. Therefore, we evaluate how fine-tuning with samples from the target dataset improves SDLog's ability to detect sensitive attributes in RQ3. To this end, we begin with the base models trained during RQ2 (i.e., models trained on all datasets except the target one), and further fine-tune them using a small, labeled subset from the target dataset. To examine the impacts of fine-tuning sample sizes, we consider three fine-tuning sample sizes: 20, 50, and 100 lines of logs.

To simulate the arrival sequence of log messages, for each log data in our general dataset, we reserve the first 20, 50, and 100 logs that each contain at least one sensitive attribute as the fine-tuning set. For unified evaluation, the remaining 1,900 logs excluded from the 100 sampled lines are used as the fixed test set. For example, in RQ2,

Just Accepted

Table 8. Performance of fine-tuned SDLog for individual target datasets (with 20, 50, and 100 logs) in detecting sensitive attributes across the general dataset. The highest F1 values are highlighted in bold.

Dataset	0			20			50			100		
	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)
Android	80.5	89.9	84.9	92.8	99.0	**95.8**	88.0	99.0	93.6	86.7	100.0	92.9
Apache	100.0	100.0	**100.0**	100.0	100.0	**100.0**	100.0	100.0	**100.0**	100.0	100.0	**100.0**
BGL	100.0	86.3	92.6	85.7	92.3	88.9	98.5	100.0	**99.2**	85.5	100.0	92.2
Hadoop	98.8	80.0	88.4	80.6	84.5	82.5	81.4	92.5	86.8	82.8	99.6	**90.4**
HDFS	93.1	95.1	94.1	100.0	100.0	**100.0**	100.0	100.0	**100.0**	100.0	100.0	**100.0**
HPC	100.0	68.8	81.5	97.1	98.1	97.6	54.3	98.1	69.9	96.3	100.0	**98.1**
Linux	99.8	99.7	99.7	99.9	99.6	**99.8**	99.5	99.6	99.6	99.6	99.6	99.6
Mac	62.3	49.9	55.4	67.0	58.5	62.5	68.3	85.6	76.0	91.1	96.2	**93.6**
OpenSSH	92.0	91.6	91.8	100.0	99.7	99.8	100.0	99.8	**99.9**	100.0	99.8	**99.9**
OpenStack	100.0	88.8	94.1	100.0	88.7	94.0	100.0	95.0	97.4	100.0	99.8	**99.9**
Proxifier	100.0	100.0	**100.0**	100.0	100.0	**100.0**	100.0	100.0	**100.0**	100.0	100.0	**100.0**
Spark	62.3	64.2	63.2	82.6	97.9	89.6	99.7	97.9	**98.8**	97.0	97.9	97.5
Thunderbird	97.1	86.5	91.5	96.9	87.7	92.1	98.1	87.6	92.6	92.4	95.3	**93.8**
Windows	99.8	99.0	99.4	100.0	99.2	**99.6**	100.0	99.2	**99.6**	100.0	99.2	**99.6**
Zookeeper	99.9	99.8	**99.8**	85.4	100.0	92.1	91.7	99.9	95.6	88.8	99.9	94.0
Macro Average	92.4	86.6	89.1	92.5	93.7	93.0	92.0	96.9	93.9	94.7	99.2	**96.8**
Micro Average	94.6	91.2	92.9	96.2	96.3	96.3	96.9	98.1	97.5	97.4	99.5	**98.4**

Table 9. Performance of fine-tuned SDLog for individual target datasets (with 20, 50, and 100 logs) in categorizing sensitive attributes on the general dataset. The highest F1 values are highlighted in bold.

Attribute	0			20			50			100		
	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)
Net	99.5	97.8	**98.6**	96.5	99.5	98.0	96.1	99.8	97.9	96.9	99.9	98.4
MAC	100.0	40.0	57.1	94.7	38.3	54.5	100.0	74.5	85.4	92.2	100.0	**95.9**
File path	99.9	94.8	97.3	99.1	95.9	97.5	99.4	98.3	98.8	99.4	98.9	**99.1**
ID	86.0	91.5	88.7	95.4	99.4	97.4	96.8	99.8	98.3	97.6	99.9	**98.7**
URL	0.0	0.0	0.0	100.0	62.5	76.9	100.0	62.5	76.9	99.0	88.4	**93.4**
Username	99.8	73.7	84.8	94.2	98.0	96.1	99.8	98.0	**98.9**	97.5	99.4	98.4
Configuration	95.7	34.2	50.4	97.2	29.6	45.4	96.7	62.7	76.1	96.5	93.7	**95.1**

to evaluate SDLog on the *Apache* dataset, we trained the model on the other 15 datasets and tested it on all 2,000 *Apache* logs. In this research question, we extend that setup by additionally fine-tuning the pre-trained SDLog model using 20, 50, or 100 *Apache* logs before testing it, each time using the same fixed 1,900-log test set for consistency. For this RQ, we exclude the *HealthApp* dataset from our evaluation setup, as it does not contain any sensitive attributes in the entire dataset, and therefore, it cannot be used for fine-tuning SDLog. We do not fine-tune SDLog on the net dataset, as its performance on this dataset is already near-perfect, leaving little room for meaningful improvement.

This fine-tuning and evaluation process is repeated for each dataset in the general dataset. At the end of this process, we report the impact of different fine-tuning sizes (20, 50, and 100 logs) on the model's performance. Similar to RQ2, we evaluate both its ability to detect the presence of sensitive information and its accuracy in categorizing the specific type of each sensitive attribute.

Just Accepted

Table 10. Average fine-tuning and inference time across different target dataset sizes.

Target Dataset Size (log lines)	20	50	100	500	1000	2000
Fine-tuning Time (s)	13.14 ± 1.16	15.53 ± 1.66	23.62 ± 2.70	88.77 ± 28.01	144.84 ± 59.25	205.65 ± 147.89
Inference Time (s)	5.21 ± 2.00	5.89 ± 0.91	6.99 ± 0.86	15.03 ± 1.89	24.13 ± 2.41	35.15 ± 4.00

4.3.2 Results.

Fine-tuning SDLog with target-specific data enhances its sensitive attribute detection performance. Table 8 summarizes the results of sensitive attribute detection across the general dataset when SDLog is fine-tuned using 20, 50, and 100 labeled log samples from the target dataset. In general, increasing the number of fine-tuning examples leads to improved performance. Specifically, in 14 out of 15 datasets, fine-tuning with 100 logs results in equal or higher F1-scores compared no fine-tuning (i.e., using plain SDLog) and in 13 out of 15 datasets, fine-tuning with 100 logs results in equal or higher F1-scores compared to using only 20 logs. Notably, when fine-tuned with 100 samples, SDLog achieves an F1-score above 90% and a recall above 95% on all datasets. This indicates that, across the entire general dataset, SDLog can accurately detect at least 95% of all sensitive attributes when fine-tuned with just 100 logs. Overall, using 100 fine-tuning logs, SDLog achieves an average precision of 97.4% (up by 1.2% compared to using 20 logs), a recall of 99.5% (up by 3.2% compared to using 20 logs), and an F1-score of 98.4% (a 2.1% improvement compared to using 20 logs). In some datasets, the highest F1-score is achieved with a smaller number of fine-tuning samples. For instance, BGL reaches its peak performance when fine-tuned with 50 log samples, while Android achieves its best result with only 20 samples. Notably, the optimal fine-tuning configuration yields an F1-score of 95% or higher across all datasets. These findings suggest that identifying the optimal number of fine-tuning samples requires experimenting with different data sizes to determine the performance peak.

Fine-tuning SDLog with target-specific logs significantly improves the categorization of sensitive attributes. Table 9 shows the categorization performance of SDLog after fine-tuning with varying amounts of labeled data from the target dataset. The results show that fine-tuning greatly enhances the model's ability to distinguish between different types of sensitive attributes. As the number of fine-tuning examples increases, from 20 to 100 logs, SDLog demonstrates consistent improvement gains across all attribute types. Even for attributes that already had high performance using 20 logs as fine-tuning, such as net, file path, ID, and username, fine-tuning with 100 logs yields additional improvements in F1-score ranging from 0.4% to 2.3%. More substantial gains are observed in challenging attributes. For instance, comparing the 20 log samples and 100 log samples for fine-tuning, the F1-score for URL increases from 76.9% to 93.4% (a 16.5% improvement), for MAC address from 54.5% to 95.9% (a 41.4% boost), and for configuration details, the most irregular and unstructured category, from 45.4% to 95.1% (a 49.7% increase). Notably, with just 20 fine-tuning logs, 4 out of 7 attributes already surpass a 90% F1-score, and when using 100 logs, all 7 attributes exceed 90% F1-score, highlighting the strong impact of even modest fine-tuning on SDLog's classification capabilities.

SDLog can be efficiently integrated into CI/CD pipelines. Table 10 presents the average and standard deviation of fine-tuning and inference time of SDLog for different target dataset sizes, ranging from 20 to 2,000 log lines. Fine-tuning time increases with dataset size: approximately 13 seconds for 20 logs, 24 seconds for 100 logs, and 206 seconds for 2,000 logs. Inference time (i.e., detecting all sensitive attributes in the dataset) also scales with dataset size, taking roughly 5 seconds for 20 logs, 7 seconds for 100 logs, and 35 seconds for 2,000 logs. These results, in particular, the relatively short inference time, indicate that SDLog is lightweight and fast enough to be practically deployed for automated sensitive attribute detection (i.e., model inference) in continuous integration and deployment workflows.

Just Accepted

Summary of RQ3

Our results show that fine-tuning SDLog on a small, labeled subset of the target dataset significantly improves its performance in both detecting and categorizing sensitive attributes, and slightly larger fine-tuning sample sets can bring more effectiveness improvements. With as few as 100 fine-tuning samples, SDLog accurately identifies 99.5% of sensitive attributes in the general dataset and achieves an F1-score of 98.4% on average. Fine-tuning also enhances the model's ability to classify specific attribute types, with special improvements for more challenging categories such as configuration details. When fine-tuned with 100 samples, SDLog reaches an F1-score exceeding 90% across all categories of sensitive attributes. These findings highlight the practical value of lightweight fine-tuning in adapting SDLog to new environments and improving its effectiveness.

5 Discussion

5.1 When are regular expressions suitable for detecting sensitive attributes in software logs?

Combining results from our first two research questions, we identify several inherent limitations of regular expressions for detecting sensitive attributes in software logs. First, there is no standardized ground truth for regex patterns and their performance varies widely; for example, we found 17 distinct patterns for identifying IP addresses, a relatively well-defined format, and some corresponding regexes gained extremely low F1-scores (e.g., 0.0%, 1.1%, and 9.1%). Second, even small syntactic differences between similar-looking regexes can lead to large performance variations. Third, although regex patterns can effectively detect some well-formatted patterns (e.g., IP addresses), they perform poorly on attributes that have no defined format and are context-dependent, such as username, ID, and configuration details. These attributes are often ignored or weakly captured in practice. Finally, in real-world anonymization pipelines where multiple regex patterns are executed sequentially, overall performance is highly sensitive to their ordering, which must be carefully optimized using domain knowledge. This sensitivity is especially pronounced for structurally composite attributes such as URLs, which frequently embed other entities (e.g., usernames, IPs, IDs, or ports), creating interdependencies between patterns. Although it is possible to design structurally aware regexes that explicitly account for such overlaps, doing so requires extensive manual analysis of attribute relationships and constraints, thereby limiting scalability and increasing configuration effort compared to SDLog's data-driven approach. An additional challenge in using regex for sensitive attribute detection is the incompatibility among frameworks. For instance, native Unicode property escapes (\P{...}) were not supported until JavaScript ES2018. In this study, we use Python 3's *re* module as the regex engine for all experiments.

Despite these limitations, regexes can still be effective in certain controlled scenarios. Our results show that when the most effective patterns are carefully selected and applied in an optimal sequence, regexes can perform well on attributes with clearly defined structures, such as MAC addresses or URLs, achieving F1-scores above 95%. However, for more ambiguous attributes such as username, configuration details, or ID, regexes remain unreliable even under ideal conditions.

5.2 What are the advantages of using a learning-based model such as SDLog for detecting sensitive attributes in software logs?

Adopting a deep learning model such as SDLog offers several key advantages over traditional regex-based methods. Unlike regexes, SDLog does not rely on hand-crafted patterns and can identify a wide range of sensitive attributes, including those that lack rigid or well-defined formats (e.g., usernames, IDs, and configuration details). Additionally, SDLog eliminates the problem of dependencies on the order of rules. Since it uses an unified

Just Accepted

model for all attribute types, it avoids conflicts and interferences that often arise when multiple regexes are applied sequentially. Furthermore, SDLog benefits from contextual understanding, allowing it to detect sensitive information based not just on format, but also on surrounding semantics, something regex patterns are inherently incapable of.

Compared to large language models such as GPT-5 and Llama3, SDLog is specifically fine-tuned on diverse log datasets, enabling it to better capture the structural and contextual characteristics of software logs. In contrast, GPT-5 and Llama3 are general-purpose language models trained primarily on natural language and broad web data, which may not align well with the rigid syntax and domain-specific conventions of log messages. As demonstrated in Section 4.2.2, SDLog substantially outperforms these models in detecting sensitive attributes, achieving significantly higher F1-scores across datasets. This performance gap highlights the importance of domain adaptation: models tailored to the log domain can more effectively distinguish structured entities and context-dependent attributes, whereas general-purpose LLMs tend to overgeneralize or misclassify tokens when applied to specialized technical text. Overall, our findings suggest that targeted fine-tuning on domain-specific data provides a more reliable and efficient solution for sensitive attribute detection in real-world logging environments.

SDLog provides a pre-trained solution that can generalize well to unseen data: it can serve as a pre-trained model for sensitive attribute detection in software logs, eliminating the need for developers to manually design and validate regex patterns for every new environment. It also provides a fine-tuning opportunity when a small batch of labeled data (e.g., 100 lines of logs) from the target system is available. The small batch labeled data can further enhance SDLog's system-specific performance. In summary, SDLog offers a more robust, flexible, and scalable solution for sensitive data detection in software logs.

5.3 How can SDLog be integrated into real-world log anonymization pipelines?

Sensitive attribute detection and categorization is an essential stage before log anonymization, since each type of sensitive attribute has different corresponding anonymization approaches. As addressed in Section 2.3, existing anonymization approaches often leverage regex patterns for attribute detection, which have critical limitations on scope and accuracy. In comparison, our plain pre-trained SDLog outperforms the best-performing regular expressions and flagship large language models such as GPT-5. Moreover, SDLog can obtain even better performance when fine-tuned on as few as 20 to 100 labeled log lines from the target dataset.

SDLog aims to provide effective sensitive attribute detection performance and facilitate the downstream anonymization task. Integrating SDLog into an anonymization pipeline is straightforward: users can simply replace the regex-based module in their anonymization framework with SDLog for attribute identification. Apart from anonymization, SDLog can also be used for re-identification risk measurement. For instance, practitioners can use SDLog to extract the sensitive attributes from their log files and public datasets, then calculate re-identification risk metrics (e.g., k-anonymity, differential privacy). With the evaluation outcome, one can make a more sensible decision on sharing log files. The pre-trained version of SDLog is publicly available on Hugging Face (see Section 8), and can be used like any standard pre-trained language model such as BERT. After formatting (i.e., tokenizing with whitespaces) the logs appropriately, developers can run SDLog with a simple API call. To support adoption, we also provide the scripts needed to prepare the raw dataset and fine-tune SDLog. Organizations and practitioners can efficiently adapt SDLog using a relatively small set of labeled examples. As shown in Section 4.3.2, both fine-tuning and inference require minimal time, making SDLog well-suited for integration into organizational CI/CD pipelines. By releasing our models and tools, we aim to make SDLog an accessible and practical alternative to regex-based methods in real-world anonymization pipelines.

Just Accepted

6 Threats to Validity

Internal validity. The definition of *sensitive* information may vary across organizations, individuals, and legal jurisdictions. To mitigate this, we adopted the definition proposed by Aghili et al. [3], which combines perspectives on sensitivity from academic literature, industrial practice, and privacy regulations. While we acknowledge that BERT and its variants may also be able to achieve good attribute detection performance after the fine-tuning process, we selected CodeBERT considering its strong alignment with our coding context.

External validity. Although we evaluated SDLog on 16 publicly available log datasets that have been widely used in prior research on software logs [36, 37, 79], the results may not generalize to all logging environments, such as those used in enterprise or government environments. Nonetheless, we demonstrated that fine-tuning SDLog on a small portion of a new dataset leads to substantial performance gains, suggesting the method is adaptable to unseen domains.

Additionally, SDLog may be less effective at detecting sensitive attributes that were not present in the training data, such as geolocation coordinates or API keys. Similarly, plain SDLog may struggle with attributes that have very limited training samples (e.g., URL, MAC address). However, our fine-tuning experiments show that even attributes that were underrepresented during pre-training can be accurately detected after domain-specific fine-tuning.

Construct validity. The ground truth annotations were created manually by the first two authors. They may include subjective judgment regarding what constitutes sensitive information. To mitigate potential bias, we followed a systematic annotation approach and measured inter-rater agreement, achieving a high reliability after two rounds of independent labeling followed by discussion sessions.

Another potential threat lies in the selection of regex patterns. We acknowledge that other powerful regex patterns for sensitive variable detection may be available online (e.g., regex patterns used along with log processing frameworks such as Logstash), but we are not able to collect and examine all these resources due to the required manual effort. Therefore, the regexes included in our paper may not reflect the most effective rules an experienced practitioner could design. To mitigate this, we systematically collected patterns from three academic databases and integrated regexes from three industry collaborators. We found a total of 41 regex patterns across eight sensitive attribute types and benchmarked only the top-performing patterns in our comparisons.

There is also a risk of overfitting, particularly when SDLog is fine-tuned on small datasets, potentially causing it to learn dataset-specific artifacts. We addressed this by employing an early stopping strategy during training to prevent overfitting.

Just Accepted

7 Conclusion

We introduce SDLog, a deep learning-based framework for detecting sensitive attributes in software logs, a critical first step for log anonymization. While traditional approaches rely heavily on manually designed regular expressions, these methods suffer from limited generalizability and poor performance on unstructured or context-dependent attributes. SDLog overcomes these limitations by leveraging contextual understanding through pre-trained language models, enabling it to generalize across diverse log formats and structures. In our evaluation across 16 datasets, SDLog consistently outperforms the baselines in identifying sensitive information. When fine-tuned with as few as 100 samples from a target dataset, SDLog achieves near-perfect detection, correctly identifying 99.5% of sensitive attributes. Our approach could be integrated into real-world log anonymization pipelines conveniently. Future work can further enhance SDLog's capabilities by incorporating additional log datasets and increasing coverage for underrepresented attribute types.

8 Data Availability

To facilitate the integration of SDLog into log anonymization pipelines, we release our models, datasets, and supporting scripts. Both the SDLog Main and SDLog Net models are hosted on Hugging Face: https://huggingface.co/LogSensitiveResearcher/SDLog_main and https://huggingface.co/LogSensitiveResearcher/SDLog_net. In addition, we provide the annotated dataset of 32,000 log lines used to train SDLog, along with scripts for dataset preparation and fine-tuning. These resources are available on our GitHub repository: https://github.com/mooselab/SDLog.

Acknowledgments

We would like to gratefully thank the Natural Sciences and Engineering Research Council of Canada (NSERC, #RGPIN-2021-03900) and the Department of Computer and Software Engineering at Polytechnique Montreal (BSFD funds) for their funding support for this work.

Just Accepted

References

[1] Roozbeh Aghili, Heng Li, and Foutse Khomh. 2023. Studying the characteristics of AIOps projects on GitHub. *Empirical Software Engineering* 28, 6 (2023), 143.

[2] Roozbeh Aghili, Heng Li, and Foutse Khomh. 2025. Protecting Privacy in Software Logs: What Should Be Anonymized? *Proceedings of the ACM on Software Engineering* 2, FSE (2025).

[3] Roozbeh Aghili, Qiaolin Qin, Heng Li, and Foutse Khomh. 2024. Understanding Web Application Workloads and Their Applications: Systematic Literature Review and Characterization. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 474–486.

[4] Laith Alzubaidi, Jinglan Zhang, Amjad J Humaidi, Ayad Al-Dujaili, Ye Duan, Omran Al-Shamma, José Santamaría, Mohammed A Fadhel, Muthana Al-Amidie, and Laith Farhan. 2021. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of big Data* 8 (2021), 1–74.

[5] Ron Artstein and Massimo Poesio. 2008. Inter-coder agreement for computational linguistics. *Computational linguistics* 34, 4 (2008), 555–596.

[6] Prasasthy Balasubramanian, Justin Seby, and Panos Kostakos. 2023. Transformer-based llms in cybersecurity: An in-depth study on log anomaly detection and conversational defense mechanisms. In *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 3590–3599.

[7] Titus Barik, Robert DeLine, Steven Drucker, and Danyel Fisher. 2016. The bones of the system: A case study of logging and telemetry at microsoft. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 92–101.

[8] Victor R Basili, Richard W Selby, and David H Hutchens. 1986. Experimentation in software engineering. *IEEE Transactions on software engineering* 7 (1986), 733–743.

[9] Mohamed Amine Batoun, Mohammed Sayagh, Roozbeh Aghili, Ali Ouni, and Heng Li. 2024. A literature review and existing challenges on software logging practices: From the creation to the analysis of software logs. *Empirical Software Engineering* 29, 4 (2024), 103.

[10] Daniel M Bikel, Scott Miller, Richard Schwartz, and Ralph Weischedel. 1998. Nymble: a high-performance learning name-finder. *arXiv preprint cmp-lg/9803003* (1998).

[11] Jasmin Bogatinovski, Sasho Nedelkoski, Alexander Acker, Florian Schmidt, Thorsten Wittkopp, Soeren Becker, Jorge Cardoso, and Odej Kao. 2021. Artificial intelligence for it operations (aiops) workshop white paper. *arXiv preprint arXiv:2101.06054* (2021).

[12] Andrew Borthwick, John Sterling, Eugene Agichtein, and Ralph Grishman. 1998. Description of the MENE Named Entity System as used in MUC-7. In *Proceedings of the Seventh Message Understanding Conference (MUC-7), Fairfax, Virginia, April 29-May 1, 1998*.

[13] Genevieve Caumartin, Qiaolin Qin, Sharon Chatragadda, Janmitsinh Panjrolia, Heng Li, and Diego Elias Costa. 2025. Exploring the Potential of Llama Models in Automated Code Refinement: A Replication Study. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 681–692.

[14] An Ran Chen, Tse-Hsun Chen, and Shaowei Wang. 2021. Demystifying the challenges and benefits of analyzing user-reported logs in bug reports. *Empirical Software Engineering* 26 (2021), 1–30.

[15] Marcello Cinque, Domenico Cotroneo, and Antonio Pecchia. 2012. Event logs for the analysis of software failures: A rule-based approach. *IEEE Transactions on Software Engineering* 39, 6 (2012), 806–821.

[16] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.

[17] Hetong Dai, Yiming Tang, Heng Li, and Weiyi Shang. 2023. PILAR: Studying and mitigating the influence of configurations on log parsing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 818–829.

[18] Biplob Debnath, Mohiuddin Solaimani, Muhammad Ali Gulzar Gulzar, Nipun Arora, Cristian Lumezanu, Jianwu Xu, Bo Zong, Hui Zhang, Guofei Jiang, and Latifur Khan. 2018. LogLens: A real-time log analysis system. In *2018 IEEE 38th international conference on*

distributed computing systems (ICDCS). IEEE, 1052–1062.

[19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[20] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[21] Ulrich Flegel. 2002. Pseudonymizing Unix log files. In *International Conference on Infrastructure Security*. Springer, 162–179.

[22] Michalis Foukarakis, Demetres Antoniades, Spiros Antonatos, and Evangelos P Markatos. 2007. Flexible and high-performance anonymization of NetFlow records using anontool. In *2007 Third International Conference on Security and Privacy in Communications Networks and the Workshops-SecureComm 2007*. IEEE, 33–38.

[23] Michael Foukarakis, Demetres Antoniades, and Michalis Polychronakis. 2009. Deep packet anonymization. In *Proceedings of the Second European Workshop on System Security*. 16–21.

[24] Tadayoshi Fushiki. 2011. Estimation of prediction error by using K-fold cross-validation. *Statistics and Computing* 21 (2011), 137–146.

[25] Th Gamer, Chr Mayer, and Marcus Schöller. 2008. Pktanon–a generic framework for profile-based traffic anonymization. (2008).

[26] Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. 2016. LSTM: A search space odyssey. *IEEE transactions on neural networks and learning systems* 28, 10 (2016), 2222–2232.

[27] Xiaodan Gu and Kai Dong. 2023. PD-PAn: Prefix-and Distribution-Preserving Internet of Things Traffic Anonymization. *Electronics* 12, 20 (2023), 4369.

[28] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[29] Kilem Gwet. 2002. Kappa statistic is not satisfactory for assessing the extent of agreement between raters. *Statistical methods for inter-rater reliability assessment* 1, 6 (2002), 1–6.

[30] Chunjing Han, Kunkun Sun, Haina Tang, Yulei Wu, and Xiaodan Zhang. 2020. AFT-Anon: A scaling method for online trace anonymization based on anonymous flow tables. In *2020 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 1–7.

[31] Pinjia He, Jieming Zhu, Shilin He, Jian Li, and Michael R Lyu. 2017. Towards automated log parsing for large-scale log data analysis. *IEEE Transactions on Dependable and Secure Computing* 15, 6 (2017), 931–944.

[32] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services (ICWS)*. IEEE, 33–40.

[33] Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. 2021. Cosqa: 20,000+ web queries for code search and question answering. *arXiv preprint arXiv:2105.13239* (2021).

[34] Shaohan Huang, Yi Liu, Carol Fung, Rong He, Yining Zhao, Hailong Yang, and Zhongzhi Luan. 2020. Paddy: An event log parsing approach using dynamic dictionary. In *NOMS 2020-2020 IEEE/IFIP network operations and management symposium*. IEEE, 1–8.

[35] Hugging Face. 2025. Token Classification with Transformers. https://huggingface.co/docs/transformers/tasks/token_classification Accessed on Feb 28, 2026.

[36] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R Lyu. 2024. Lilac: Log parsing using llms with adaptive parsing cache. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 137–160.

[37] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R Lyu. 2024. A large-scale evaluation for log parsing techniques: How far are we?. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 223–234.

[38] Zhen Ming Jiang, Ahmed E Hassan, Parminder Flora, and Gilbert Hamann. 2008. Abstracting execution logs to execution events for enterprise applications (short paper). In *2008 The Eighth International Conference on Quality Software*. IEEE, 181–186.

[39] Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. 2020. Is bert really robust? a strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34-05. 8018–8025.

[40] Zanis Ali Khan, Donghwan Shin, Domenico Bianculli, and Lionel Briand. 2022. Guidelines for assessing the accuracy of log message template identification techniques. In *Proceedings of the 44th International Conference on Software Engineering*. 1095–1106.

[41] Łukasz Korzeniowski and Krzysztof Goczyła. 2022. Landscape of automated log analysis: A systematic literature review and mapping study. *IEEE Access* 10 (2022), 21892–21913.

[42] Dimitris Koukis, Spyros Antonatos, Demetres Antoniades, Evangelos P Markatos, and Panagiotis Trimintzios. 2006. A generic anonymization framework for network traffic. In *2006 IEEE International Conference on Communications*, Vol. 5. IEEE, 2302–2309.

[43] Lawrence Berkeley National Laboratory. 1998. The Internet Traffic Archive. https://ita.ee.lbl.gov/html/traces.html Accessed on Feb 28, 2026.

[44] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942* (2019).

[45] Max Landauer, Markus Wurzenberger, Florian Skopik, Giuseppe Settanni, and Peter Filzmoser. 2018. Dynamic log file analysis: An unsupervised cluster evolution approach for anomaly detection. *computers & security* 79 (2018), 94–116.

Just Accepted

[46] Heng Li, Weiyi Shang, Bram Adams, Mohammed Sayagh, and Ahmed E Hassan. 2020. A qualitative study of the benefits and costs of logging from developers' perspectives. *IEEE Transactions on Software Engineering* 47, 12 (2020), 2858–2873.
[47] Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li. 2020. A survey on deep learning for named entity recognition. *IEEE transactions on knowledge and data engineering* 34, 1 (2020), 50–70.
[48] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. 2022. SwissLog: Robust anomaly detection and localization for interleaved unstructured logs. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (2022), 2762–2780.
[49] Yufei Li, Yanchi Liu, Haoyu Wang, Zhengzhang Chen, Wei Cheng, Yuncong Chen, Wenchao Yu, Haifeng Chen, and Cong Liu. 2023. Glad: Content-aware dynamic graphs for log anomaly detection. In *2023 IEEE International Conference on Knowledge Graph (ICKG)*. IEEE, 9–18.
[50] Zhijing Li, Qiuai Fu, Zhijun Huang, Jianbo Yu, Yiqian Li, Yuanhao Lai, and Yuchi Ma. 2024. Revisiting Log Parsing: The Present, the Future, and the Uncertainties. *IEEE Transactions on Reliability* 73, 3 (2024), 1459–1472.
[51] Zhenhao Li, Chuan Luo, Tse-Hsun Chen, Weiyi Shang, Shilin He, Qingwei Lin, and Dongmei Zhang. 2023. Did we miss something important? studying and exploring variable-aware log abstraction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 830–842.
[52] Ying-Dar Lin, Po-Ching Lin, Sheng-Hao Wang, I-Wei Chen, and Yuan-Cheng Lai. 2014. Pcaplib: A system of extracting, classifying, and anonymizing real packet traces. *IEEE Systems Journal* 10, 2 (2014), 520–531.
[53] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
[54] David Lo, Nachiappan Nagappan, and Thomas Zimmermann. 2015. How practitioners perceive the relevance of software engineering research. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 415–425.
[55] Liam Daly Manocchio, Siamak Layeghy, David Gwynne, and Marius Portmann. 2024. A configurable anonymisation approach for network flow data: Balancing utility and privacy. *Computers and Electrical Engineering* 118 (2024), 109465.
[56] Ehsan Mashhadi and Hadi Hemmati. 2021. Applying codebert for automated program repair of java simple bugs. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 505–509.
[57] Andrew McCallum and Wei Li. 2003. Early results for named entity recognition with conditional random fields, feature induction and web-enhanced lexicons. In *Proceedings of the seventh conference on Natural language learning at HLT-NAACL 2003*. 188–191.
[58] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.
[59] Frank McSherry and Ratul Mahajan. 2010. Differentially-private network trace analysis. *ACM SIGCOMM Computer Communication Review* 40, 4 (2010), 123–134.
[60] Salma Messaoudi, Annibale Panichella, Domenico Bianculli, Lionel Briand, and Raimondas Sasnauskas. 2018. A search-based approach for accurate identification of log message formats. In *Proceedings of the 26th Conference on Program Comprehension*. 167–177.
[61] Haibo Mi, Huaimin Wang, Yangfan Zhou, Michael Rung-Tsong Lyu, and Hua Cai. 2013. Toward fine-grained, unsupervised, scalable performance diagnosis for production cloud computing systems. *IEEE Transactions on Parallel and Distributed Systems* 24, 6 (2013), 1245–1255.
[62] Microsoft. 2026. Presidio: Data Protection and De-identification SDK. https://microsoft.github.io/presidio/ Accessed on Feb 28, 2026.
[63] Greg Minshall. 2005. TCPDPRIV. https://ita.ee.lbl.gov/html/contrib/tcpdpriv.html Accessed on Feb 28, 2026.
[64] Seungjun Moon, Yongho Song, Hyungjoo Chae, Dongjin Kang, Taeyoon Kwon, Kai Tzu-iunn Ong, Seung-won Hwang, and Jinyoung Yeo. 2023. Coffee: Boost your code llms by fixing bugs with feedback. *arXiv preprint arXiv:2311.07215* (2023).
[65] David Nadeau and Satoshi Sekine. 2007. A survey of named entity recognition and classification. *Lingvisticae Investigationes* 30, 1 (2007), 3–26.
[66] Karthik Nagaraj, Charles Killian, and Jennifer Neville. 2012. Structured comparative analysis of systems logs to diagnose performance problems. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. 353–366.
[67] Radoslaw Naumiuk and Jarosław Legierski. 2014. Anonymization of data sets from Service Delivery Platforms. In *2014 Federated Conference on Computer Science and Information Systems*. IEEE, 955–960.
[68] Weina Niu, Zimu Li, Zhaoxu He, Aduo Wang, Beibei Li, and Xiaosong Zhang. 2023. FSMFLog: Discovering Anomalous Logs Combining Full Semantic Information and Multifeature Fusion. *IEEE Internet of Things Journal* 11, 3 (2023), 4442–4453.
[69] U.S. State of California. 2018. California Consumer Privacy Act. https://oag.ca.gov/privacy/ccpa Accessed on Feb 28, 2026.
[70] U.S. Department of Health and Human Services. 1996. Health Insurance Portability and Accountability Act. https://www.hhs.gov/hipaa/index.html Accessed on Feb 28, 2026.
[71] Momen Oqaily, Mohammad Ekramul Kabir, Suryadipta Majumdar, Yosr Jarraya, Mengyuan Zhang, Makan Pourzandi, Lingyu Wang, and Mourad Debbabi. 2023. iCAT+: An Interactive Customizable Anonymization Tool Using Automated Translation Through Deep Learning. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2023), 2799–2817.
[72] A Paszke. 2019. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703* (2019).
[73] Divya Pathak, Mudit Verma, Aishwariya Chakraborty, and Harshit Kumar. 2024. Self Adjusting Log Observability for Cloud Native Applications. In *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. IEEE, 482–493.

Just Accepted

[74] Dave Plonka. 2003. ip2anonip. https://pages.cs.wisc.edu/%7Eplonka/ip2anonip/ Accessed on Feb 28, 2026.
[75] Qiaolin Qin, Roozbeh Aghili, Heng Li, and Ettore Merlo. 2024. Preprocessing is All You Need: Boosting the Performance of Log Parsers With a General Preprocessing Framework. *arXiv preprint arXiv:2412.05254* (2024).
[76] Qiaolin Qin, Jianchen Zhao, Heng Li, Weiyi Shang, and Ettore Merlo. 2026. A Story About Cohesion and Separation: Label-Free Metric for Log Parser Evaluation. In *2026 IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE.
[77] Lance A Ramshaw and Mitchell P Marcus. 1999. Text chunking using transformation-based learning. In *Natural language processing using very large corpora*. Springer, 157–176.
[78] Piotr Ryciak, Katarzyna Wasielewska, and Artur Janicki. 2022. Anomaly detection in log files using selected natural language processing methods. *Applied Sciences* 12, 10 (2022), 5089.
[79] Shiwen Shan, Yintong Huo, Yuxin Su, Yichen Li, Dan Li, and Zibin Zheng. 2024. Face it yourselves: An llm-based two-stage strategy to localize configuration errors via logs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 13–25.
[80] Adam J Slagell, Kiran Lakkaraju, and Katherine Luo. 2006. FLAIM: A Multi-level Anonymization Framework for Computer and Network Logs.. In *LISA*, Vol. 6. 3–8.
[81] Latanya Sweeney. 2002. k-anonymity: A model for protecting privacy. *International journal of uncertainty, fuzziness and knowledge-based systems* 10, 05 (2002), 557–570.
[82] Jeniya Tabassum, Mounica Maddela, Wei Xu, and Alan Ritter. 2020. Code and Named Entity Recognition in StackOverflow. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. https://www.aclweb.org/anthology/2020.acl-main.443/
[83] Artur Varanda, Leonel Santos, Rogério Luís de C Costa, Adail Oliveira, and Carlos Rabadão. 2021. Log pseudonymization: Privacy maintenance in practice. *Journal of Information Security and Applications* 63 (2021), 103021.
[84] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A practical guide, 1st ed., Cham: Springer International Publishing* 10, 3152676 (2017), 10–5555.
[85] Zahin Wahab, Sadif Ahmed, Md Nafiu Rahman, Rifat Shahriyar, and Gias Uddin. 2024. Secret Breach Prevention in Software Issue Reports. *arXiv preprint arXiv:2410.23657* (2024).
[86] Jinyuan Wang, Tong Li, Runzi Zhang, Zifang Tang, Di Wu, and Zhen Yang. 2024. VCRLog: Variable Contents Relationship Perception for Log-based Anomaly Detection. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 156–167.
[87] Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, Guoyin Wang, and Chen Guo. 2025. Gpt-ner: Named entity recognition via large language models. In *Findings of the association for computational linguistics: NAACL 2025*. 4257–4275.
[88] Yu Wang, Hanghang Tong, Ziye Zhu, and Yun Li. 2022. Nested named entity recognition: a survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 16, 6 (2022), 1–29.
[89] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, et al. 2025. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2526–2547.
[90] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. 2012. *Experimentation in software engineering*. Vol. 236. Springer.
[91] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2019. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).
[92] Tzu-Tsung Wong. 2015. Performance evaluation of classification algorithms by k-fold and leave-one-out cross validation. *Pattern recognition* 48, 9 (2015), 2839–2846.
[93] Antonios Xenakis, Sabrina Mamtaz Nourin, Zhiyuan Chen, George Karabatis, Ahmed Aleroud, and Jhancy Amarsingh. 2023. A self-adaptive and secure approach to share network trace data. *Digital Threats: Research and Practice* 4, 4 (2023), 1–20.
[94] Jun Xu, Jinliang Fan, Mostafa H Ammar, and Sue B Moon. 2002. Prefix-preserving ip address anonymization: Measurement-based security evaluation and a new cryptography-based scheme. In *10th IEEE International Conference on Network Protocols, 2002. Proceedings.* IEEE, 280–289.
[95] Junjielong Xu, Qiuai Fu, Zhouruixing Zhu, Yutong Cheng, Zhijing Li, Yuchi Ma, and Pinjia He. 2023. Hue: A user-adaptive parser for hybrid logs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 413–424.
[96] Mukesh Yadav and Dhirendra S Mishra. 2023. Identification of network threats using live log stream analysis. In *2023 2nd International Conference on Paradigm Shifts in Communications Embedded Systems, Machine Learning and Signal Processing (PCEMS)*. IEEE, 1–6.
[97] Siyu Yu, Pinjia He, Ningjiang Chen, and Yifan Wu. 2023. Brain: Log parsing with bidirectional parallel tree. *IEEE Transactions on Services Computing* 16, 5 (2023), 3224–3237.
[98] Siyu Yu, Yifan Wu, Ying Li, and Pinjia He. 2024. Unlocking the Power of Numbers: Log Compression via Numeric Token Parsing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 919–930.

Just Accepted

[99] Siyu Yu, Yifan Wu, Zhijing Li, Pinjia He, Ningjiang Chen, and Changjian Liu. 2023. Log parsing with generalization ability under new log types. In *Proceedings of the 31st acm joint european software engineering conference and symposium on the foundations of software engineering*. 425–437.

[100] Jun Zhang and Fenfen Wang. 2022. Research on the construction of log parsing system based on regular expression. In *2022 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS)*. IEEE, 627–630.

[101] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R Lyu. 2023. Loghub: A large collection of system log datasets for ai-driven log analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 355–366.

Just Accepted